

Network Computing

(058172)



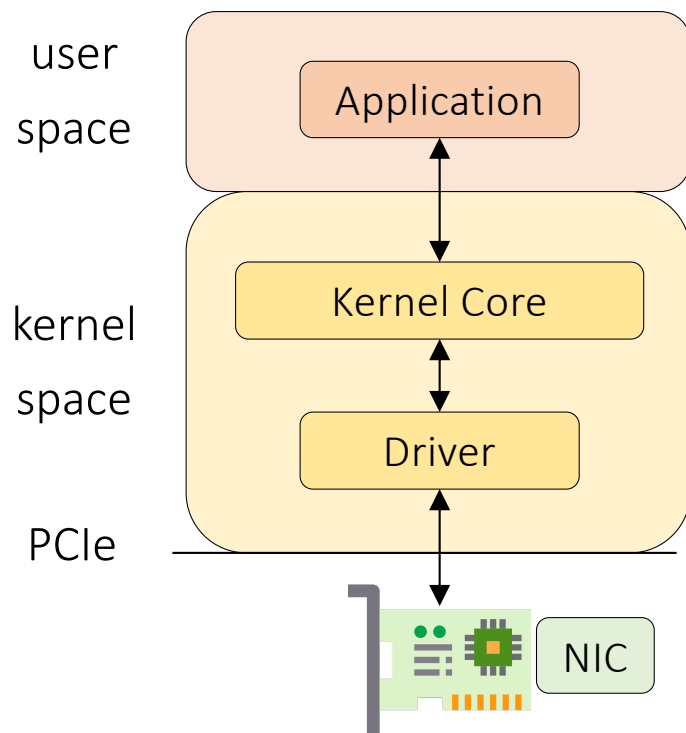
POLITECNICO
MILANO 1863

Virtual Switching

Content

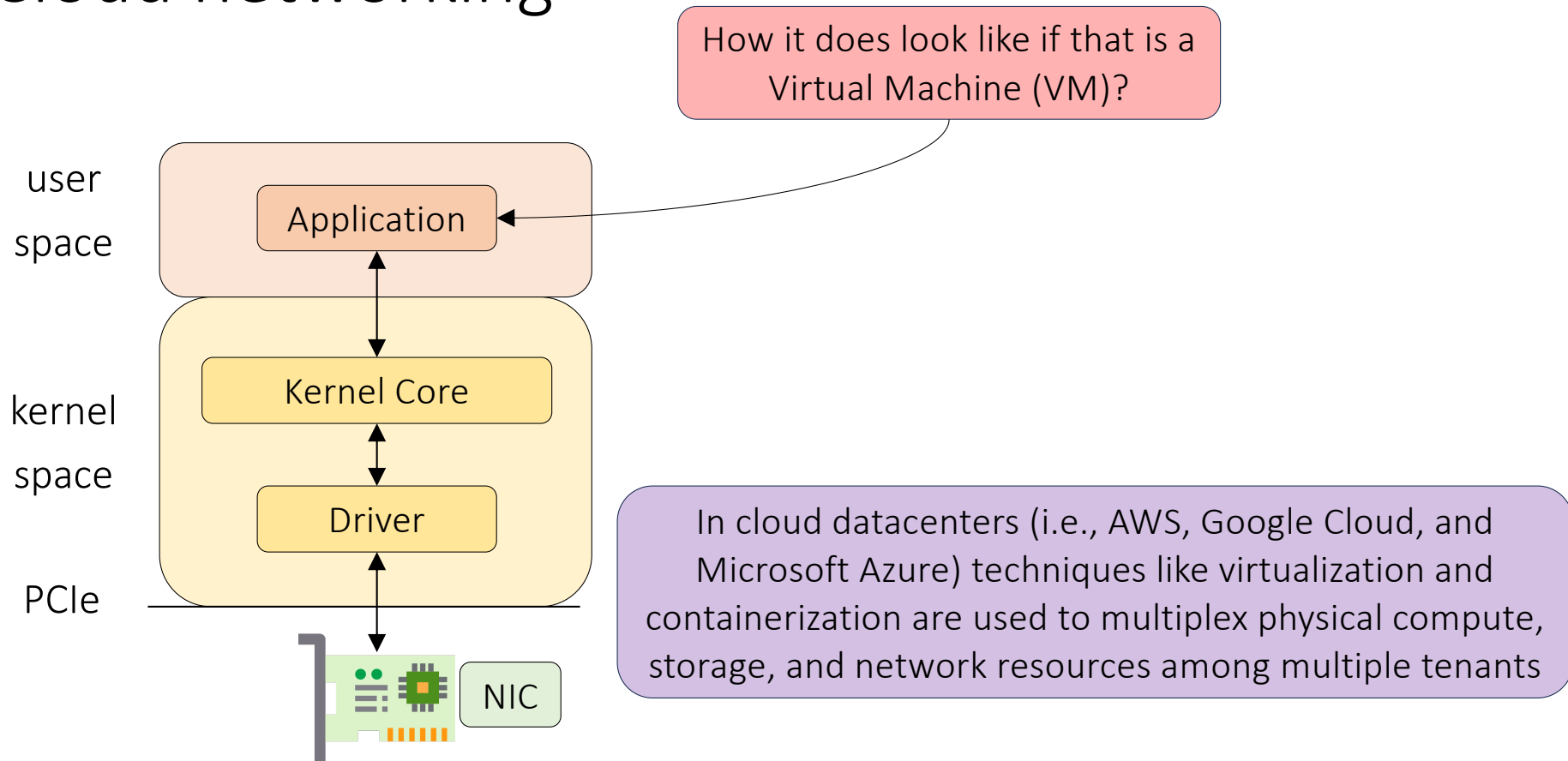
- Virtual networking in the cloud era
- Design space
- Open Virtual Switch (OVS)
- Triton

Cloud networking

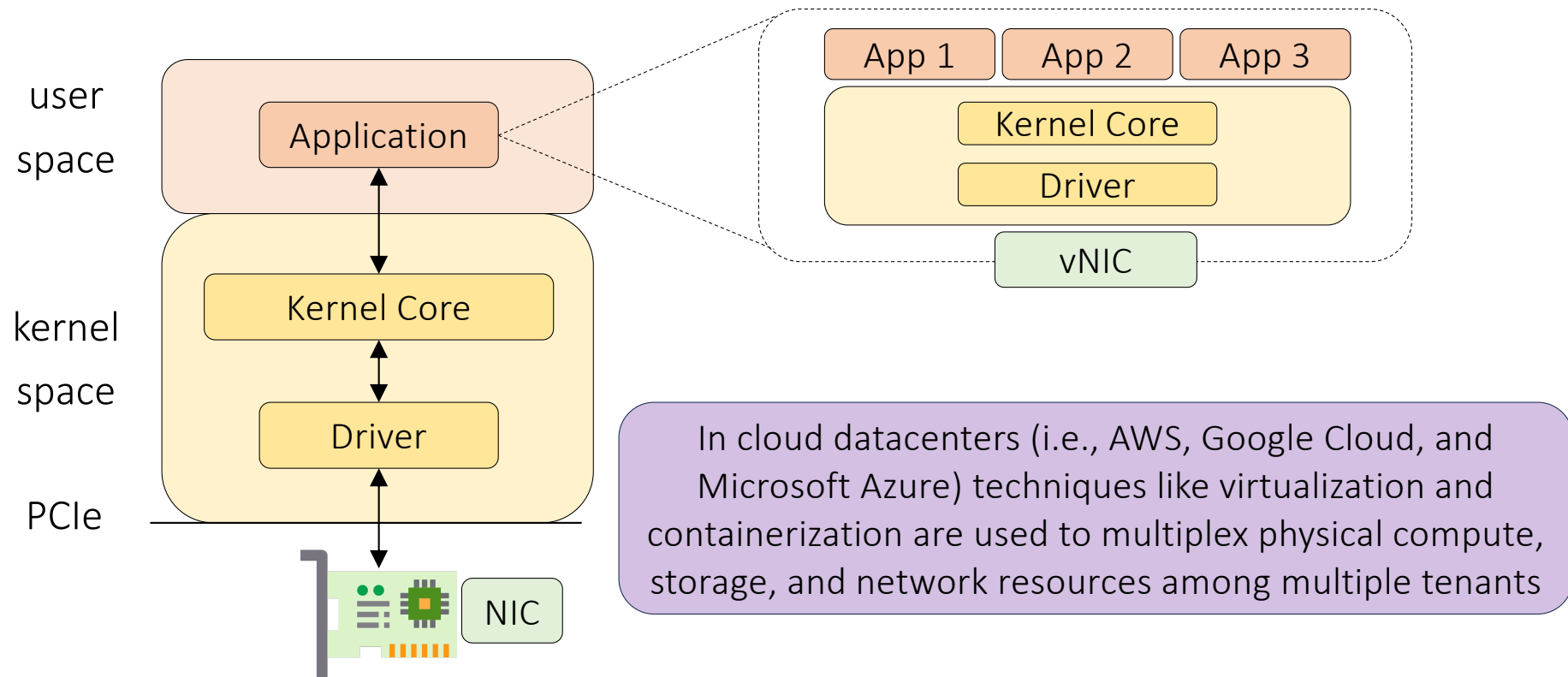


In cloud datacenters (i.e., AWS, Google Cloud, and Microsoft Azure) techniques like virtualization and containerization are used to multiplex physical compute, storage, and network resources among multiple tenants

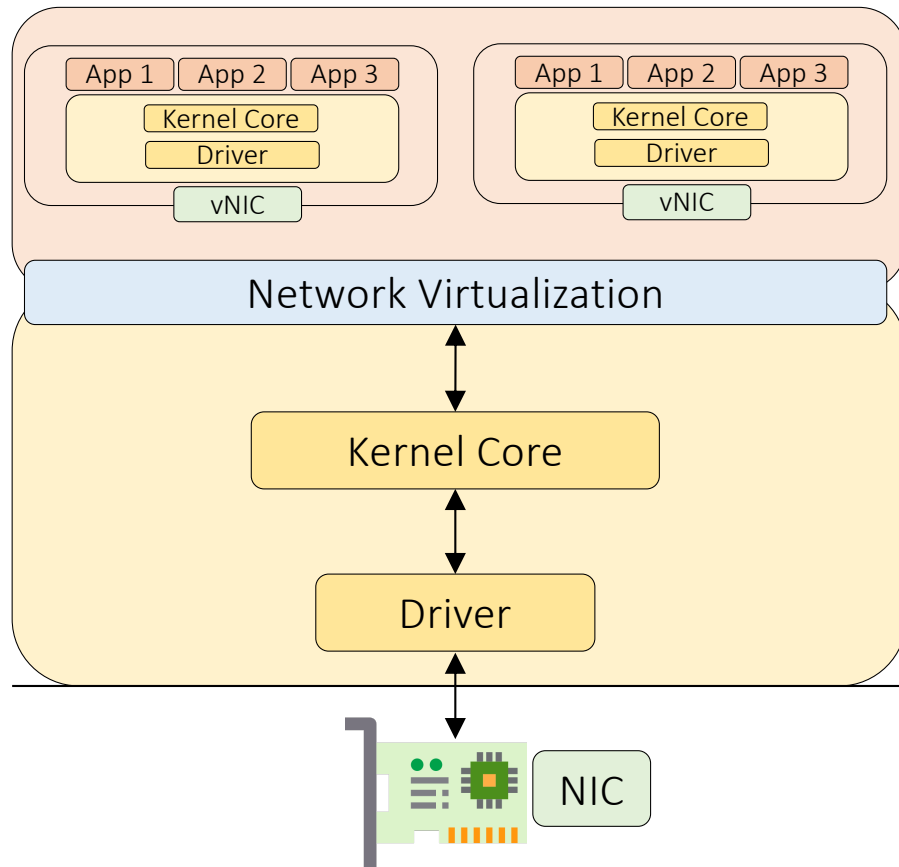
Cloud networking



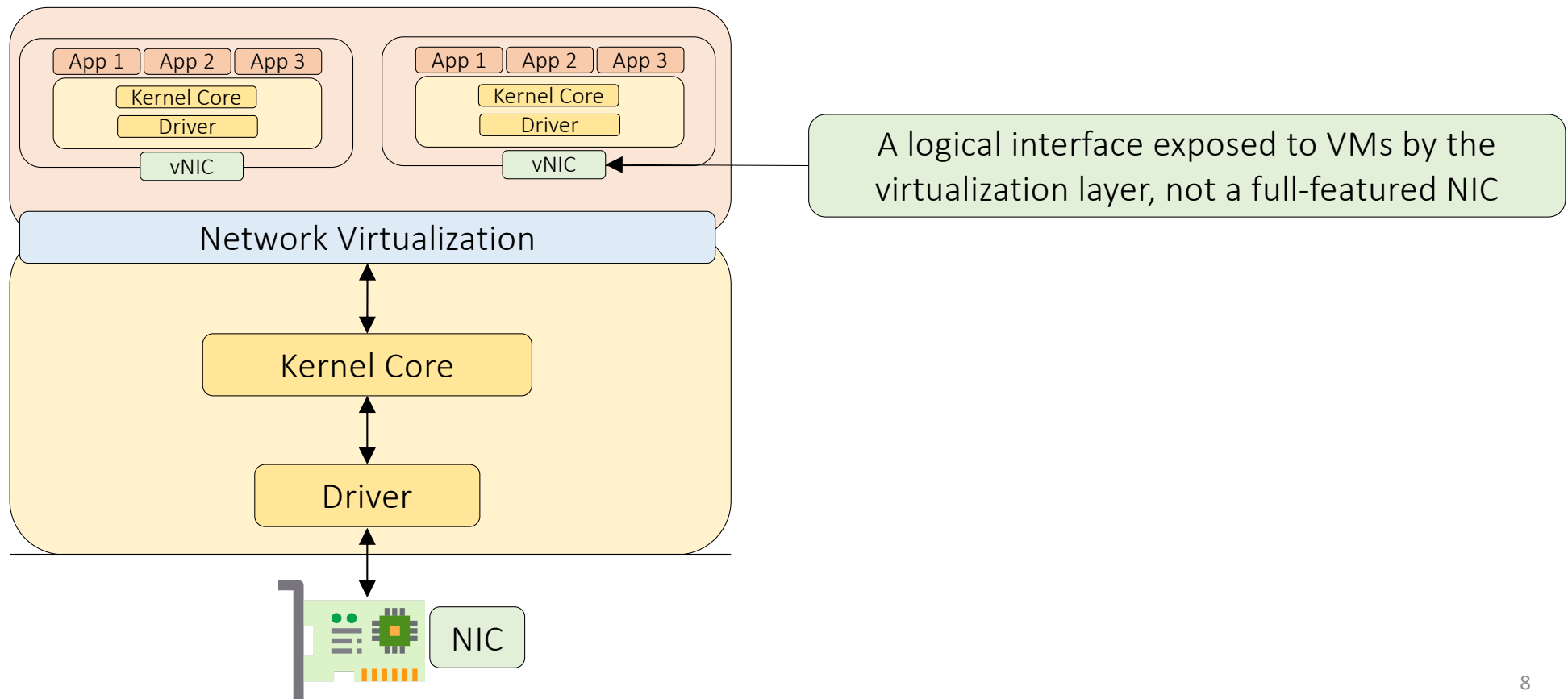
Cloud networking



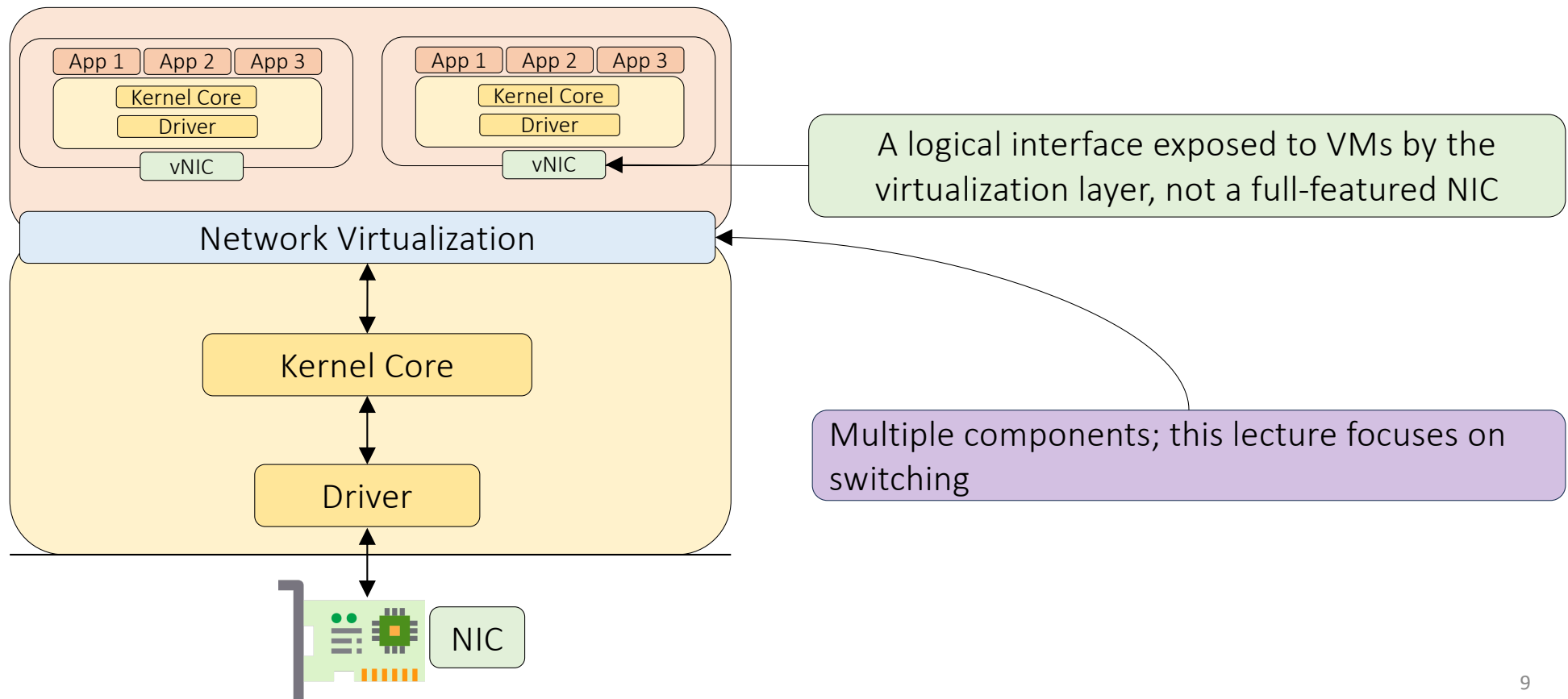
Cloud networking: putting this together



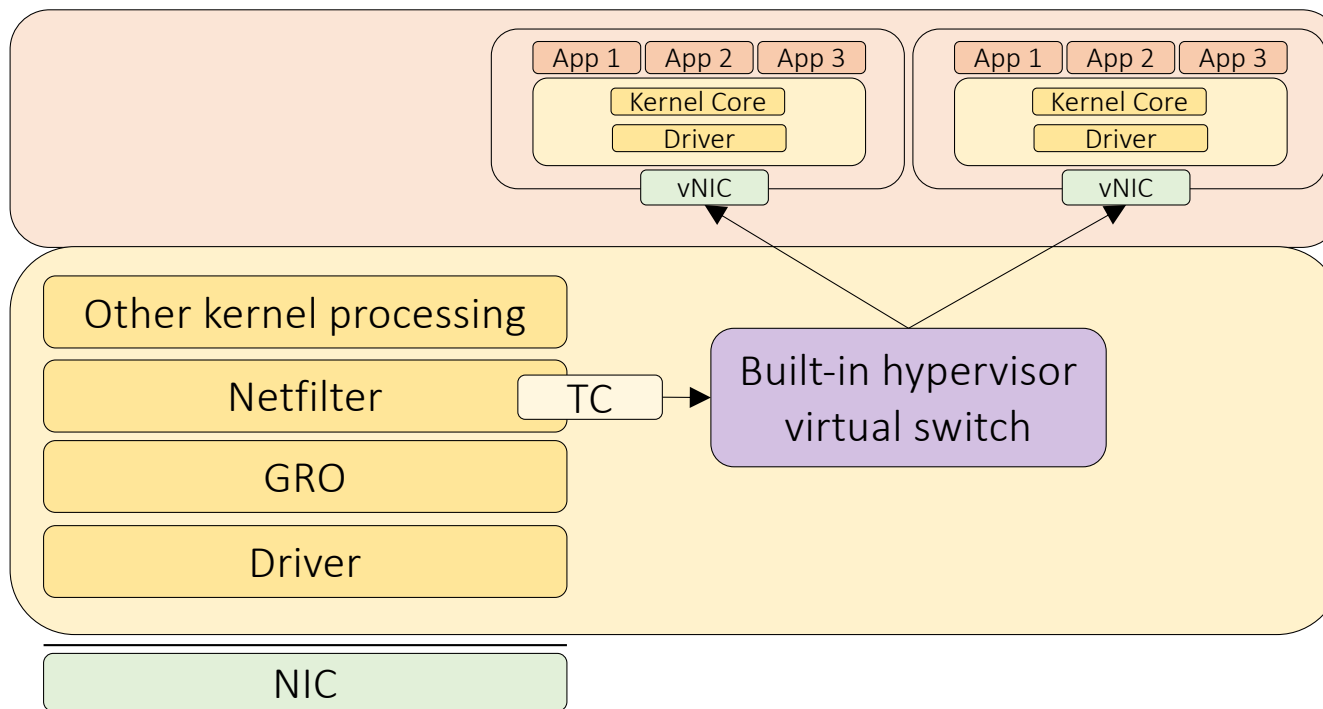
Cloud networking: putting this together



Cloud networking: putting this together



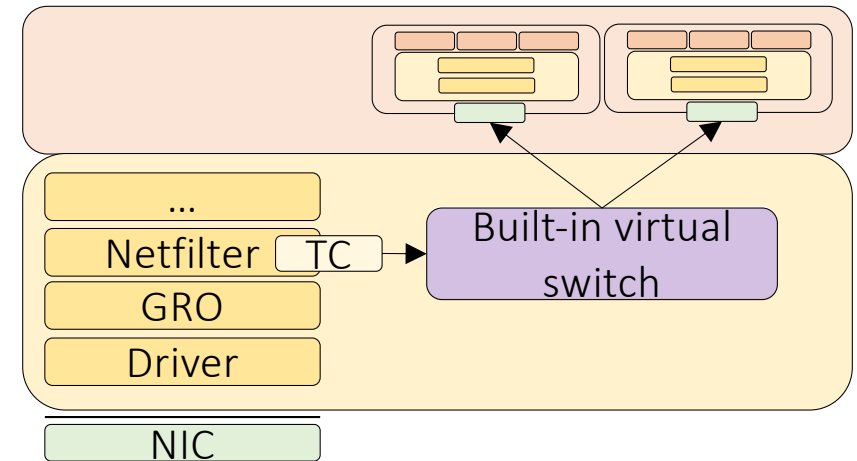
Virtual switches: the origin



The arrows show the RX path

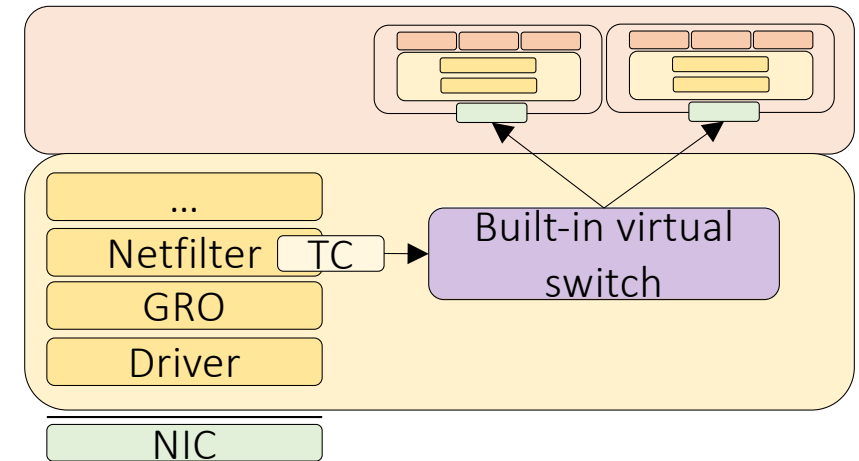
Virtual switches: the origin

No much programmability provided. It was a mechanism to statically dispatch IP packets to VMs



Virtual switches: the origin

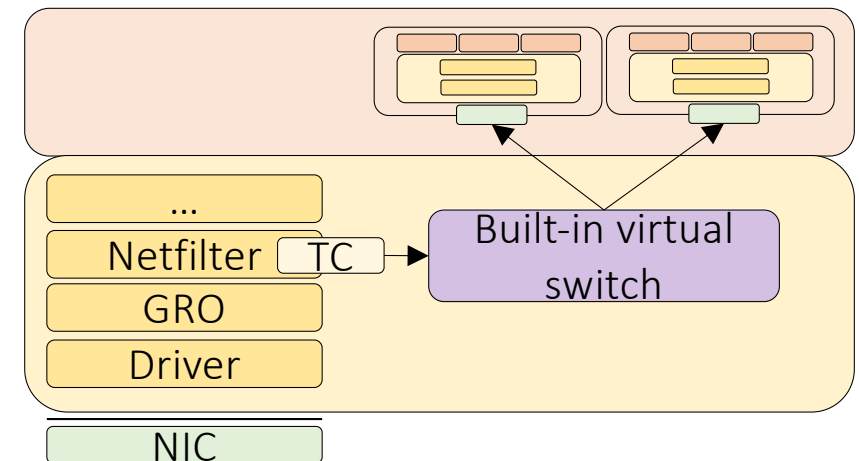
No much programmability provided. It was a mechanism to statically dispatch IP packets to VMs



Problem: in large-scale and complex virtualization environments, there is a need to operate at L2, L3 and L4 and support many advanced features (i.e., VXLAN)

Virtual switches: the origin

No much programmability provided. It was a mechanism to statically dispatch IP packets to VMs

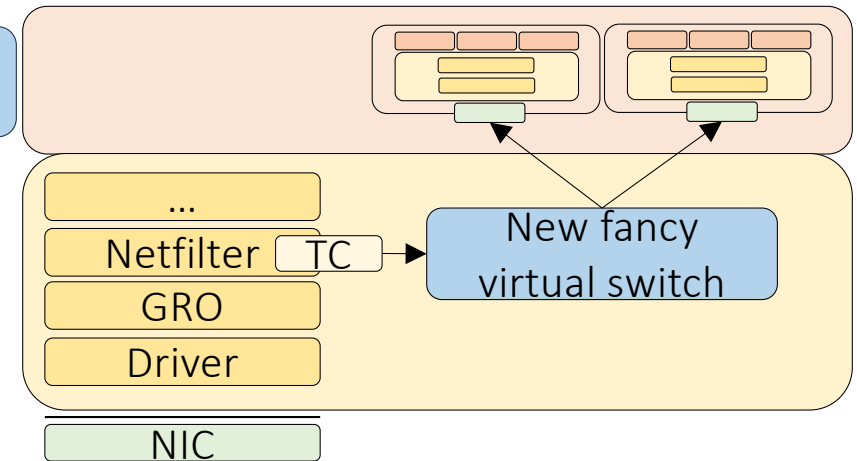


Problem: in large-scale and complex virtualization environments, there is a need to operate at L2, L3 and L4 and support many advanced features (i.e., VXLAN)

How to achieve that?

Virtual switches: design space

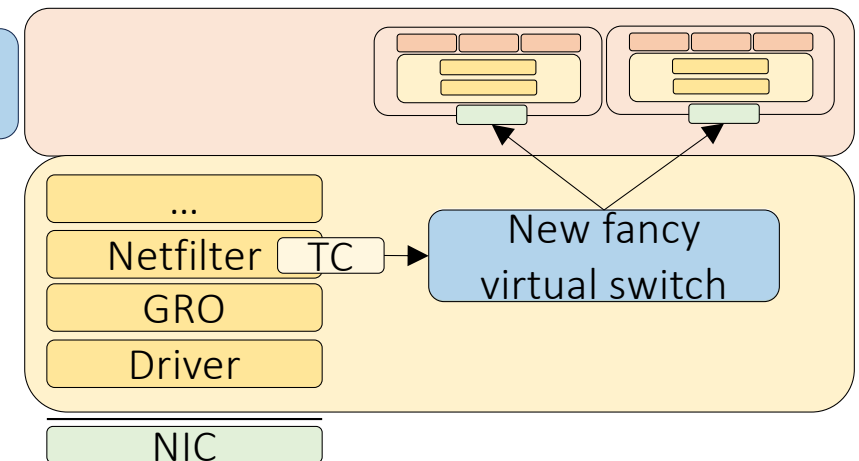
Having a custom in-kernel module running the entire virtual switch logic?



Virtual switches: design space

Having a custom in-kernel module running the entire virtual switch logic?

Performance: complex modules can impact kernel performance ☹

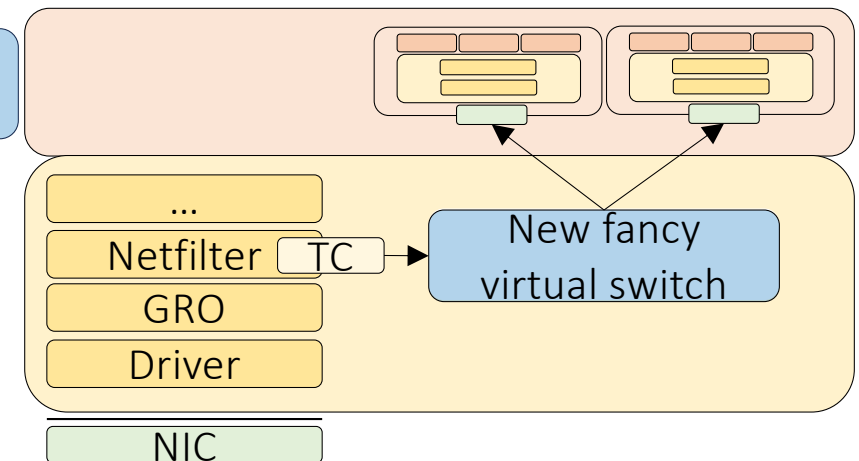


Virtual switches: design space

Having a custom in-kernel module running the entire virtual switch logic?

Performance: complex modules can impact kernel performance ☹️

Stability: frequent changes to internal kernel APIs and data structures prevent modules from being easily portable across kernel versions ☹️



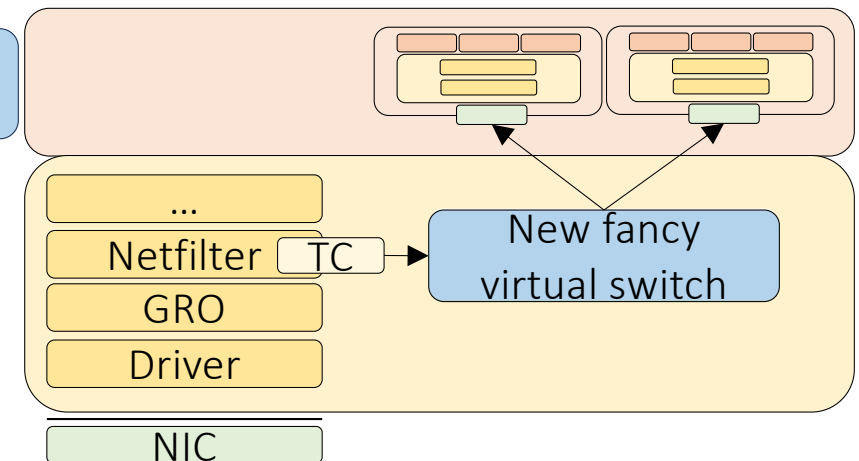
Virtual switches: design space

Having a custom in-kernel module running the entire virtual switch logic?

Performance: complex modules can impact kernel performance ☹️

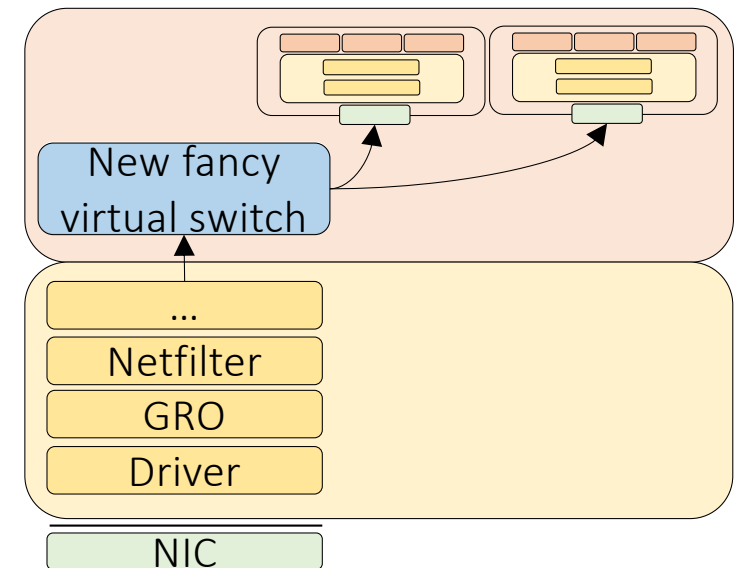
Stability: frequent changes to internal kernel APIs and data structures prevent modules from being easily portable across kernel versions ☹️

While eBPF sidesteps stable-interface issues, its sandbox limits program complexity



Virtual switches: design space

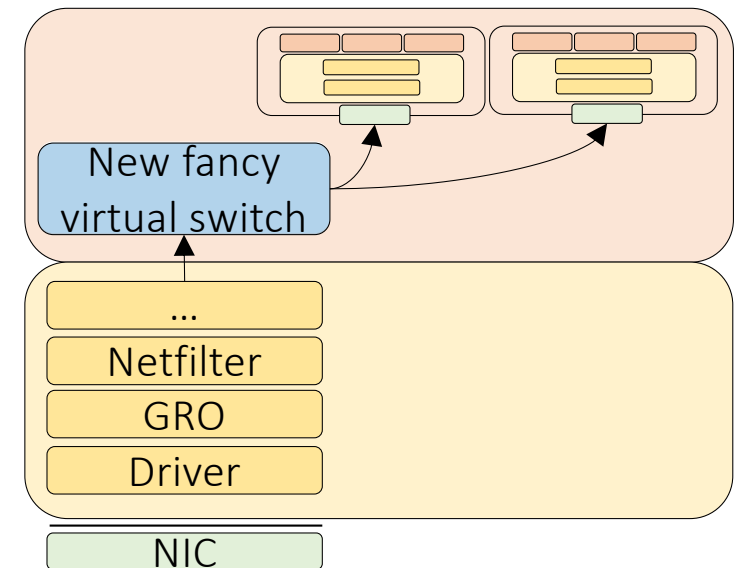
Having a custom user-space process running the entire virtual switch logic?



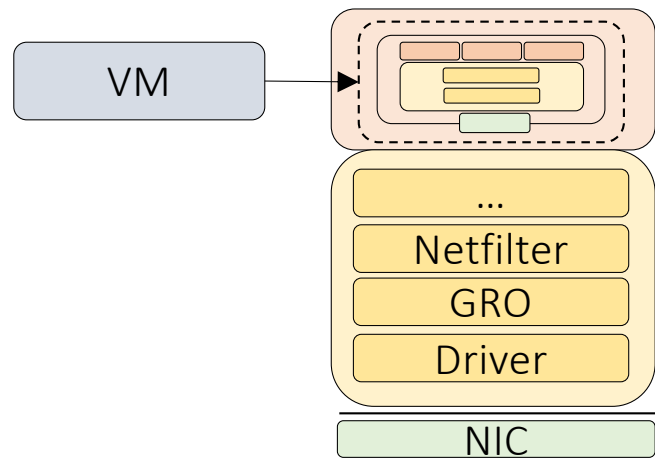
Virtual switches: design space

Having a custom user-space process running the entire virtual switch logic?

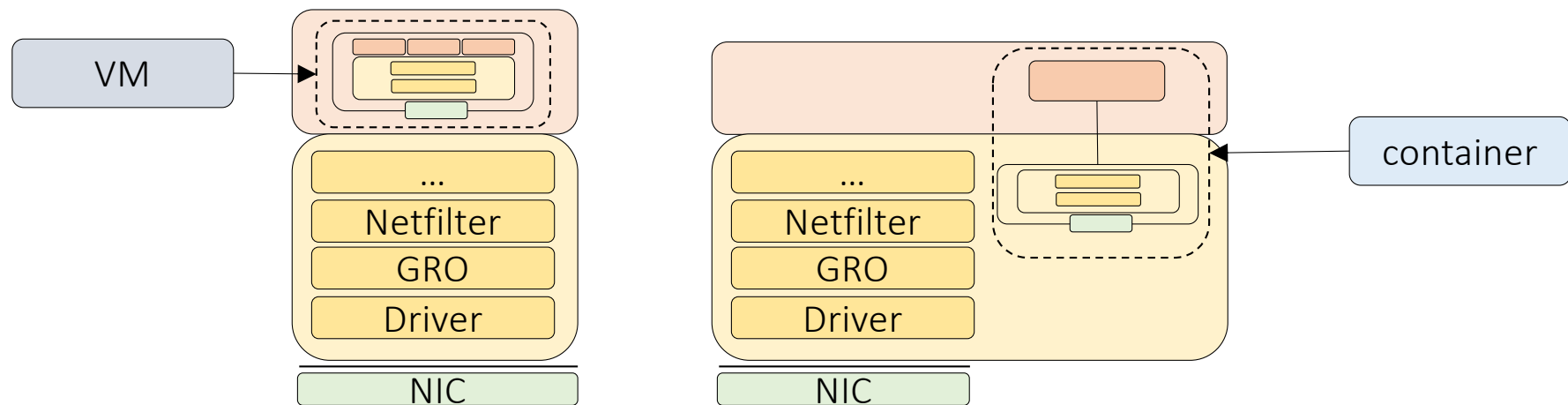
Performance: the problem is intra-host container networking



Virtual switches: design space



Virtual switches: design space

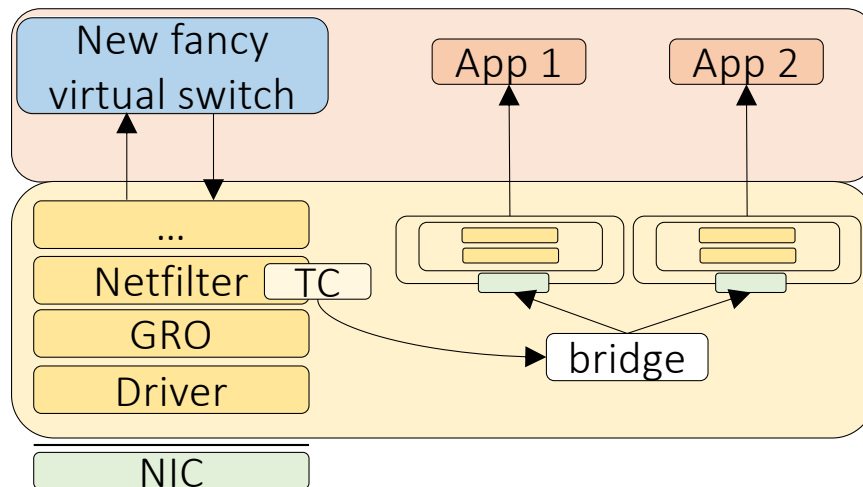


A container packages an application with its dependencies while sharing the host operating system kernel, whereas a virtual machine runs a full guest OS on virtualized hardware

Virtual switches: design space

Having a custom user-space process running the entire virtual switch logic?

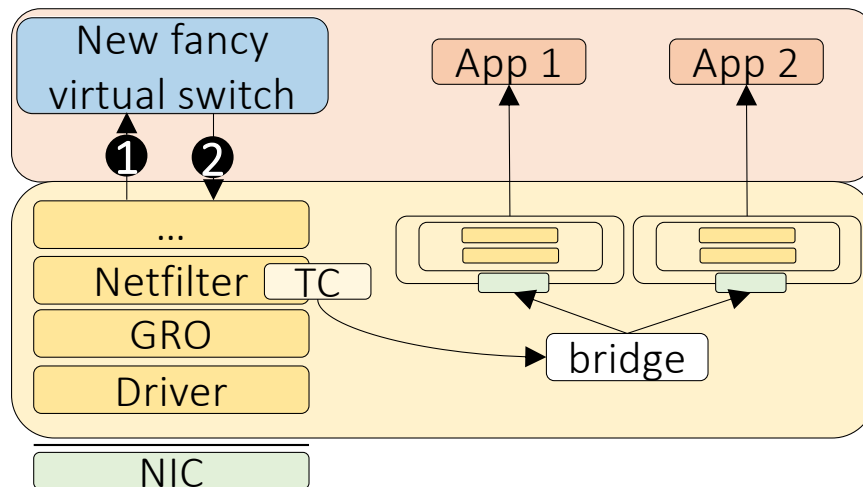
Performance: the problem is intra-host container networking



Virtual switches: design space

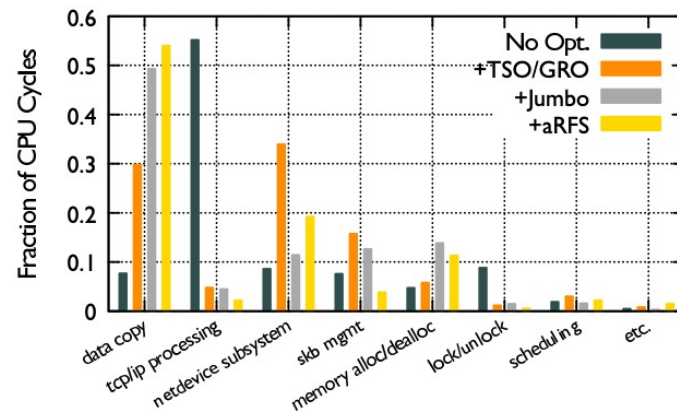
Having a custom user-space process running the entire virtual switch logic?

Performance: the problem is intra-host container networking



App-to-app communication requires two additional kernel to user-space crossings (kernel → virtual switch → kernel)

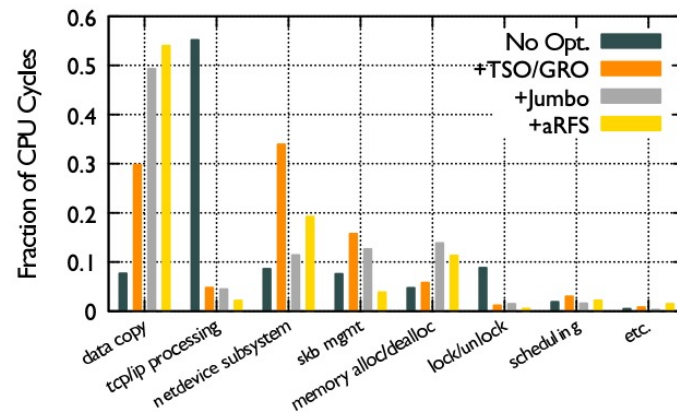
Virtual switches: design space



Data copy is the most expensive when all optimizations are used

App-to-app communication requires two additional kernel to user-space crossings (kernel → virtual switch → kernel)

Virtual switches: design space



Data copy is the most expensive when all optimizations are used

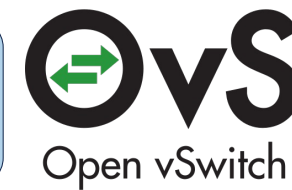
App-to-app communication requires two additional kernel to user-space crossings (kernel → virtual switch → kernel)

Other ideas? 😊

Open vSwitch



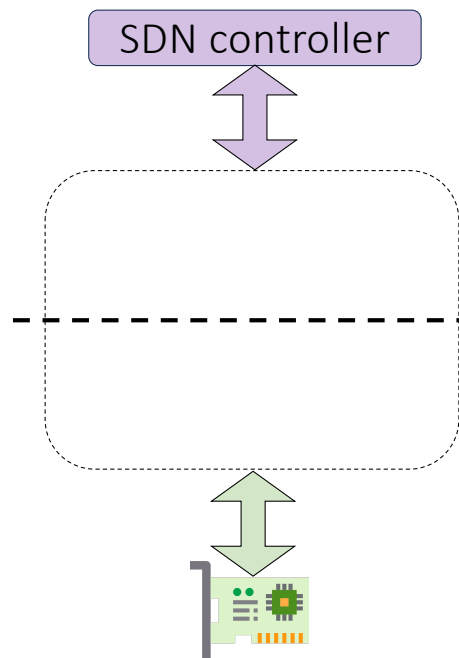
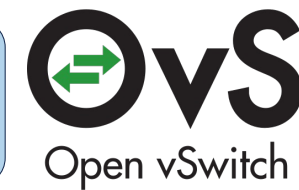
OVS is an open-source software switch that is targeted at large-scale and complex virtualization environments. It can operate at L2, L3 and L4 and supports many advanced features



Open vSwitch

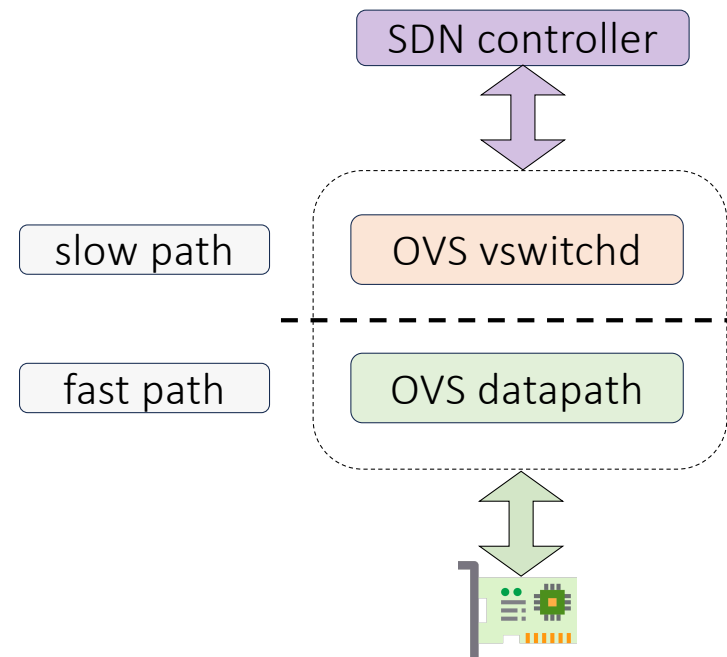
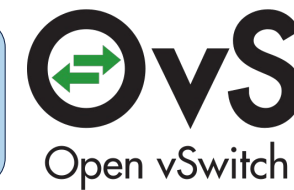


OVS is an open-source software switch that is targeted at large-scale and complex virtualization environments. It can operate at L2, L3 and L4 and supports many advanced features

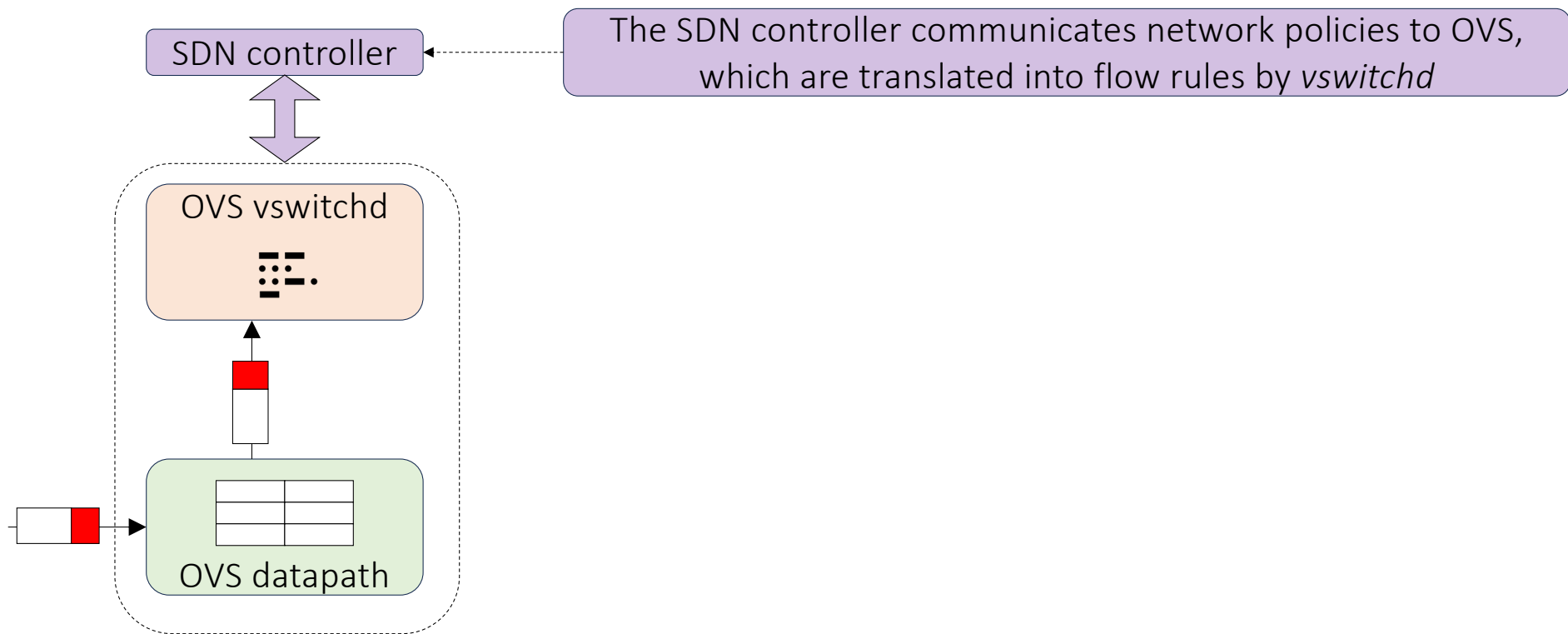


Open vSwitch

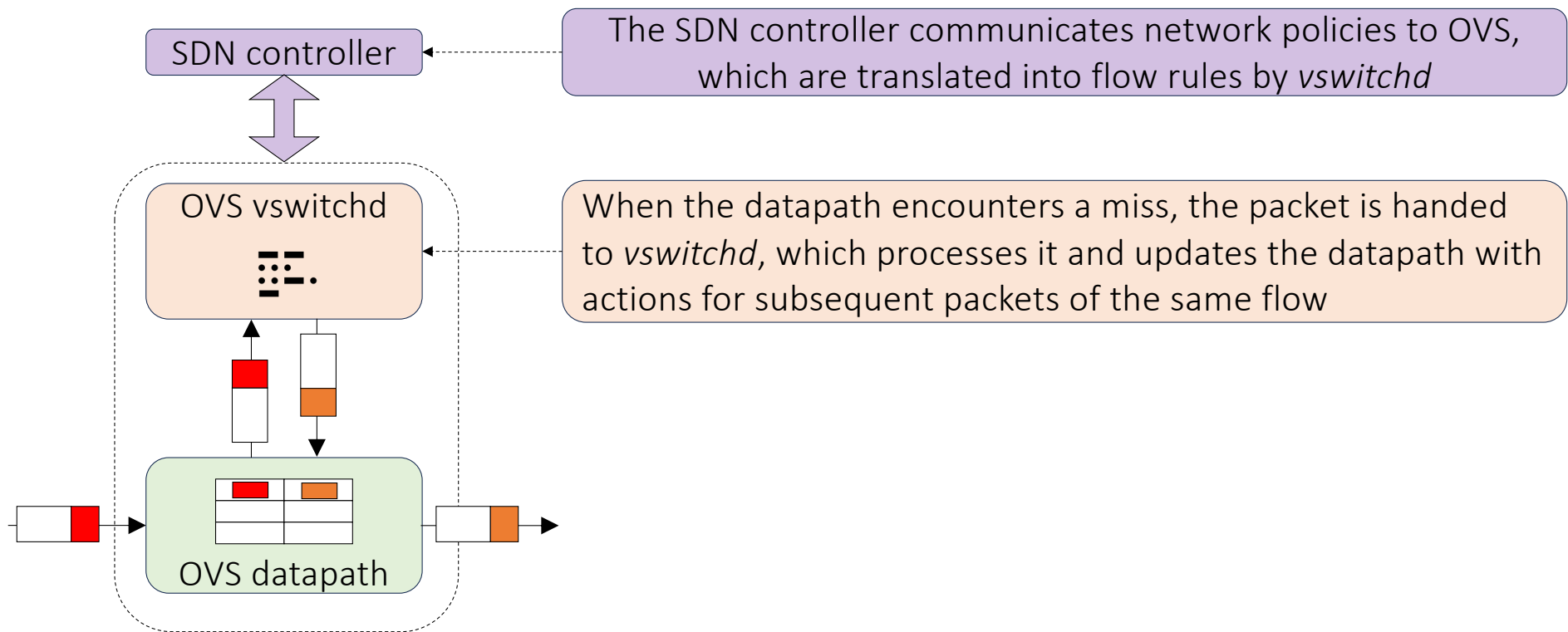
OVS is an open-source software switch that is targeted at large-scale and complex virtualization environments. It can operate at L2, L3 and L4 and supports many advanced features



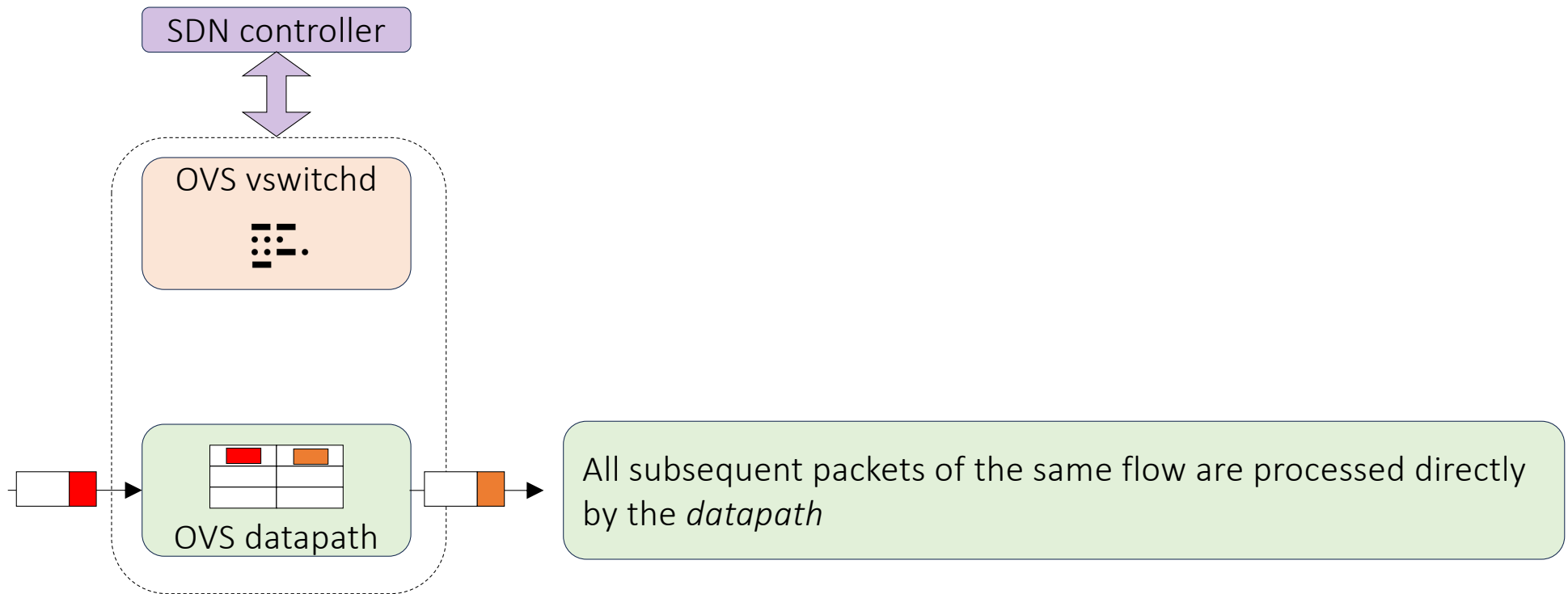
Classic Open vSwitch



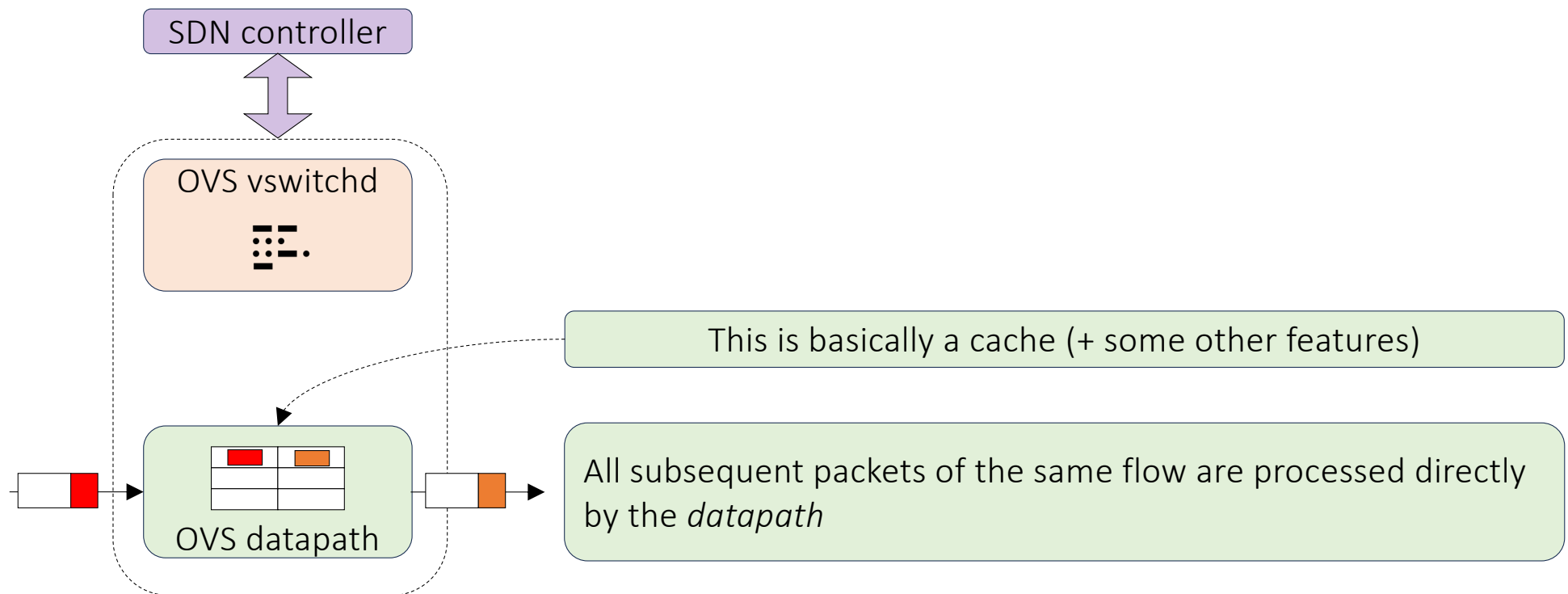
Classic Open vSwitch



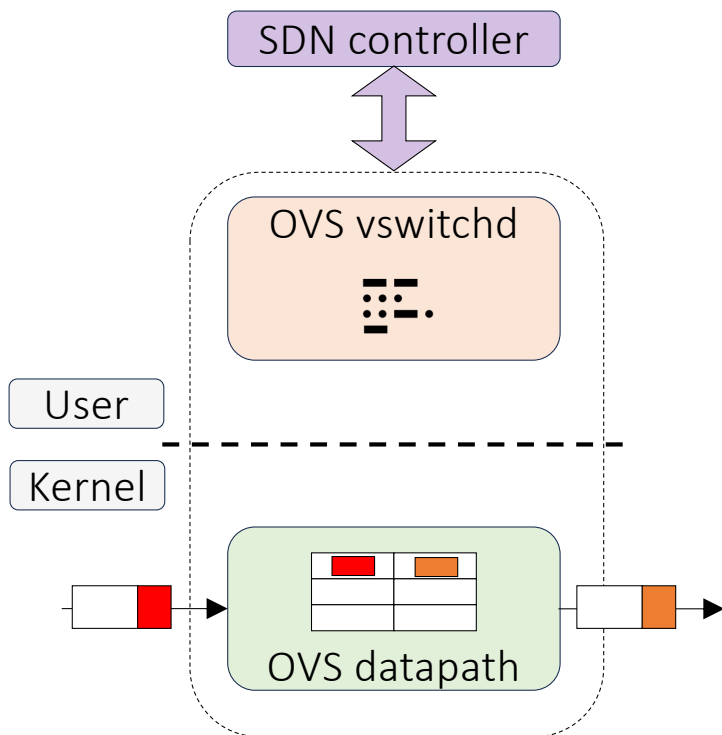
Classic Open vSwitch



Classic Open vSwitch

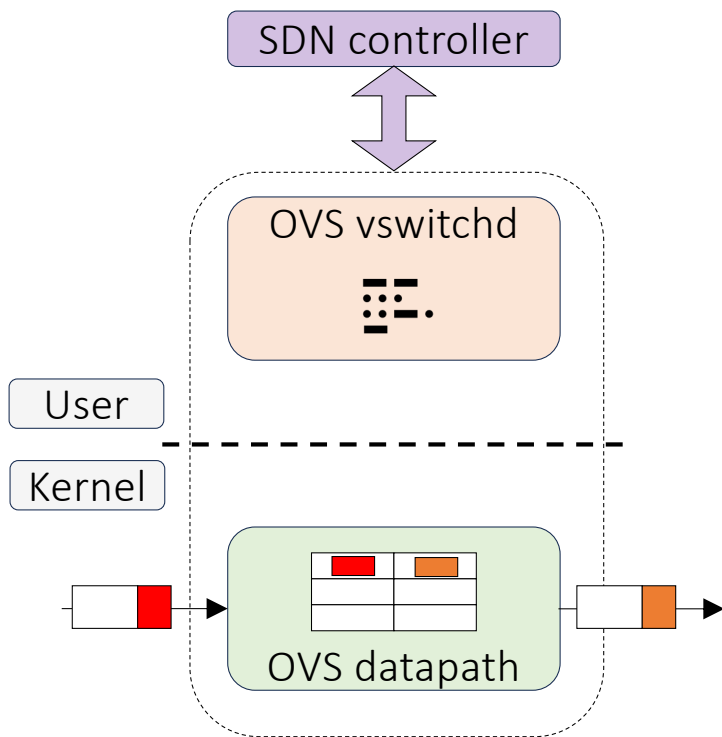


User/kernel split in OVS



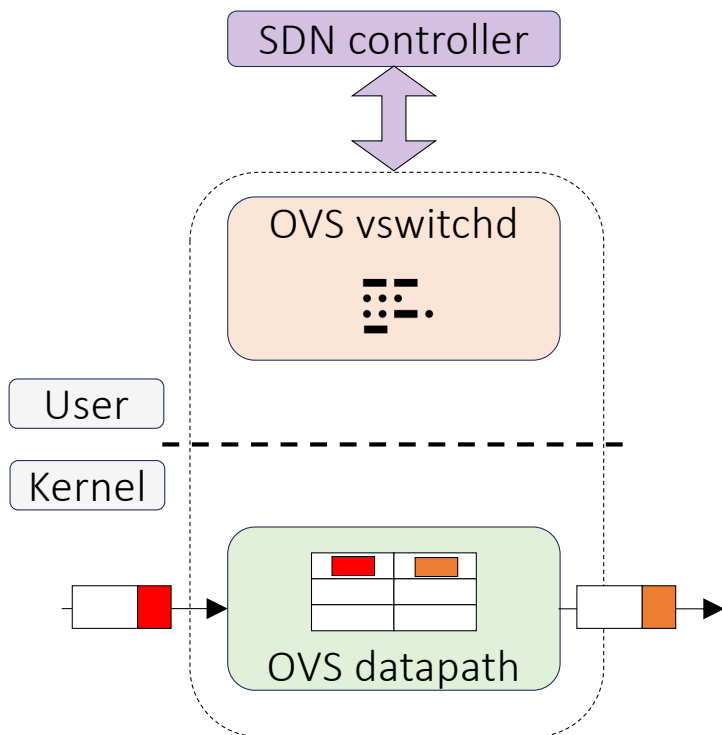
- Switching logic is implemented entirely in user space (OVS vswitchd)
- The datapath runs in the kernel, enabling efficient packet processing (e.g., for container networking)
- Because the datapath does not implement full switching logic, it can be realized as a lightweight kernel module

User/kernel split in OVS



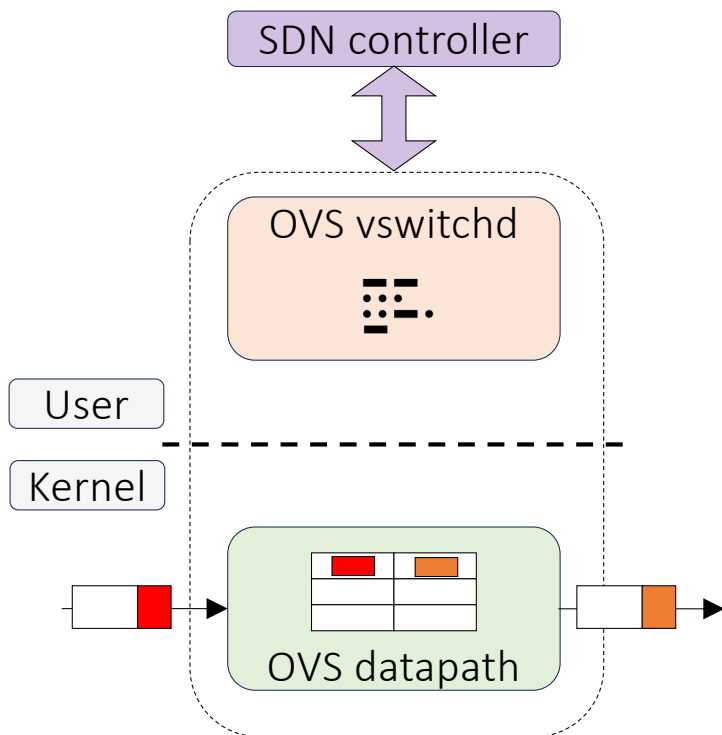
- OVS is the VMware NSX dataplane on Linux hypervisors and public clouds, and must support a wide range of Linux distributions, kernel versions, and hardware platforms
- Any new OVS feature requiring a kernel component must be accepted upstream by the Linux kernel community

User/kernel split in OVS



- The high bar of acceptance discourages features that require experimentation or iteration, and some OVS kernel features are rejected
- However, such features arise frequently, e.g., due to the introduction of new tunneling and encapsulation protocols (VXLAN, NVGRE, Geneve, NSH)

User/kernel split in OVS



- In addition, data center operators often delay upgrading kernel versions in production environments

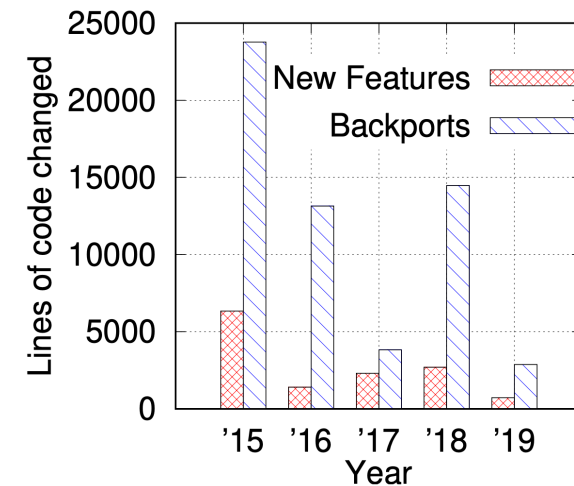
As a result, OVS features with kernel components typically reach production 1–2 years (or more) after upstream merge

Out-of-Tree kernel module

- To bypass slow upstream acceptance, OVS maintained an *out-of-tree kernel module* with newer features
- While reducing delays, this approach imposed a significant maintenance burden due to frequent kernel changes
- Even small features required substantial code changes to ensure compatibility across kernel versions

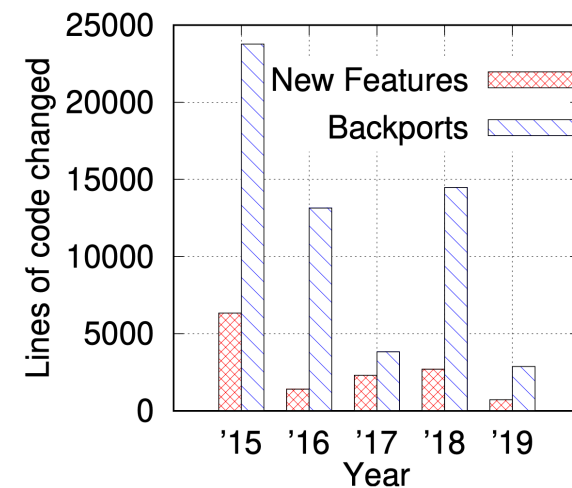
Out-of-Tree kernel module

- To bypass slow upstream acceptance, OVS maintained an *out-of-tree kernel module* with newer features
- While reducing delays, this approach imposed a significant maintenance burden due to frequent kernel changes
- Even small features required substantial code changes to ensure compatibility across kernel versions



Out-of-Tree kernel module

- To bypass slow upstream acceptance, OVS maintained an *out-of-tree kernel module* with newer features
- While reducing delays, this approach imposed a significant maintenance burden due to frequent kernel changes
- Even small features required substantial code changes to ensure compatibility across kernel versions

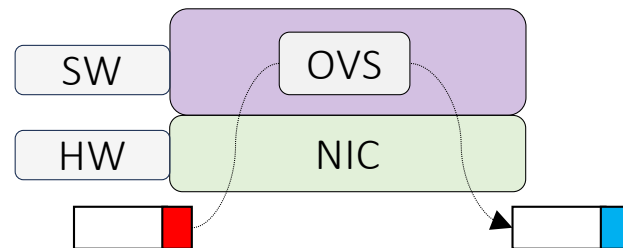


The out-of-tree module also complicated deployment due to testing across many Linux distributions and environments

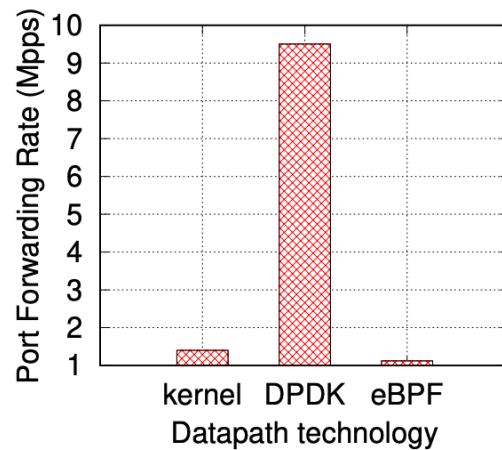
Decoupling the kernel with eBPF

- Replacing the OVS kernel module with eBPF preserves the user/kernel split while improving maintainability and deployment
- While implementing *only* the datapath makes eBPF a potential option, its sandbox constraints (e.g., program size, lack of loops) still prevent key OVS datapath functionality

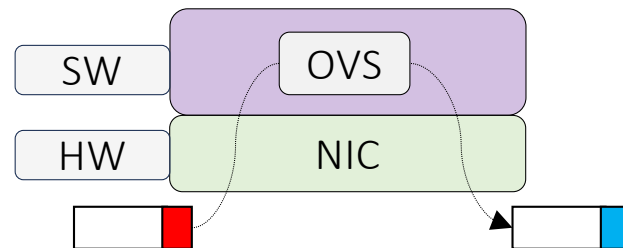
Decoupling the kernel with eBPF



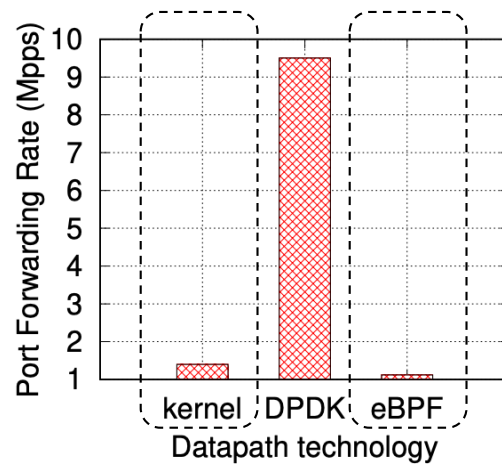
This is OVS (single core) forwarding rate for 64B packets



Decoupling the kernel with eBPF

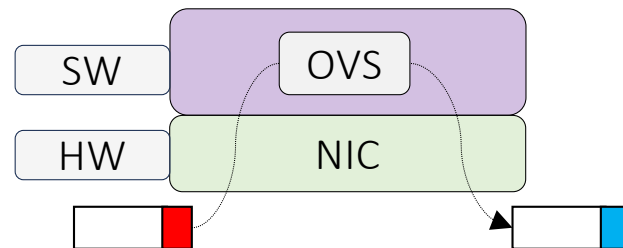


This is OVS (single core) forwarding rate for 64B packets

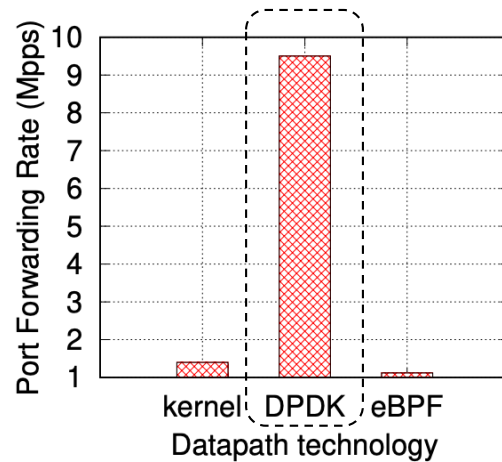


Sandboxed bytecode execution makes eBPF packet switching 10–20% slower than the native kernel module

Decoupling the kernel with eBPF



This is OVS (single core) forwarding rate for 64B packets



OVS-DPDK is fast

..but:

1. You lose all the kernel features that are important in a cloud environment
2. Operating in user space makes it less efficient for intra-host container networking

The future of OVS?

- Lately, it has been proposed a new OVS datapath built on top of AF_XDP

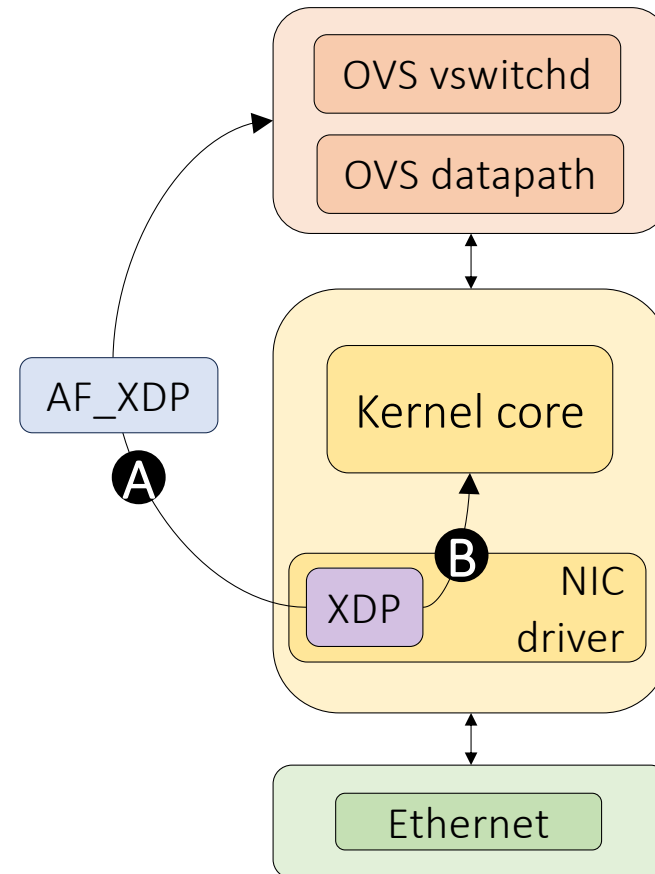
The future of OVS?

- Lately, it has been proposed a new OVS datapath built on top of AF_XDP

Why this is potentially a better architectural choice than the previous discussed?

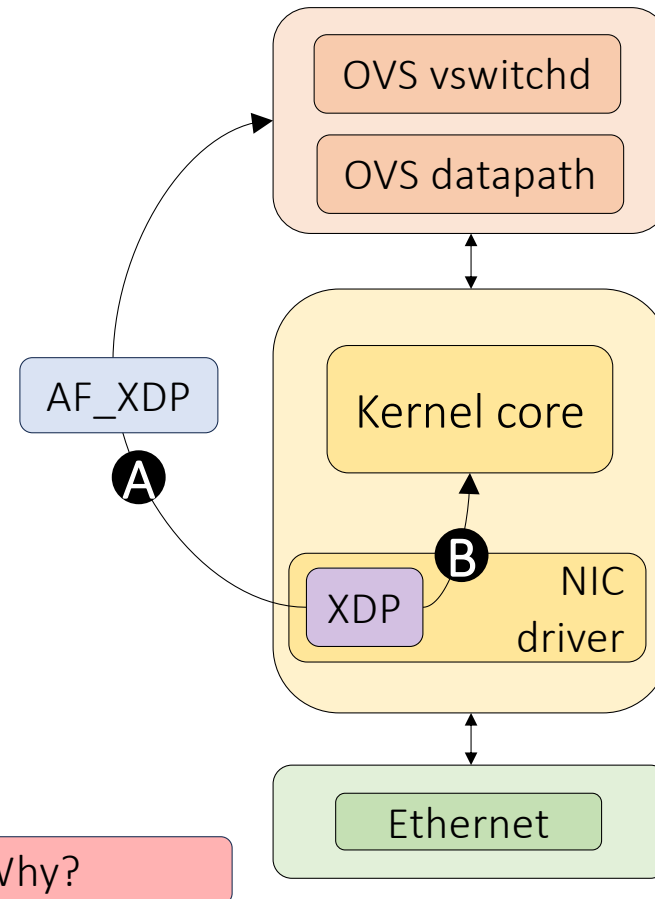
The future of OVS?

- The OVS datapath runs in user space, avoiding kernel constraints
- There are no inherent limitations on processing complexity
- Provides performance comparable to OVS-DPDK for VM networking
- Outperforms OVS-DPDK for container networking



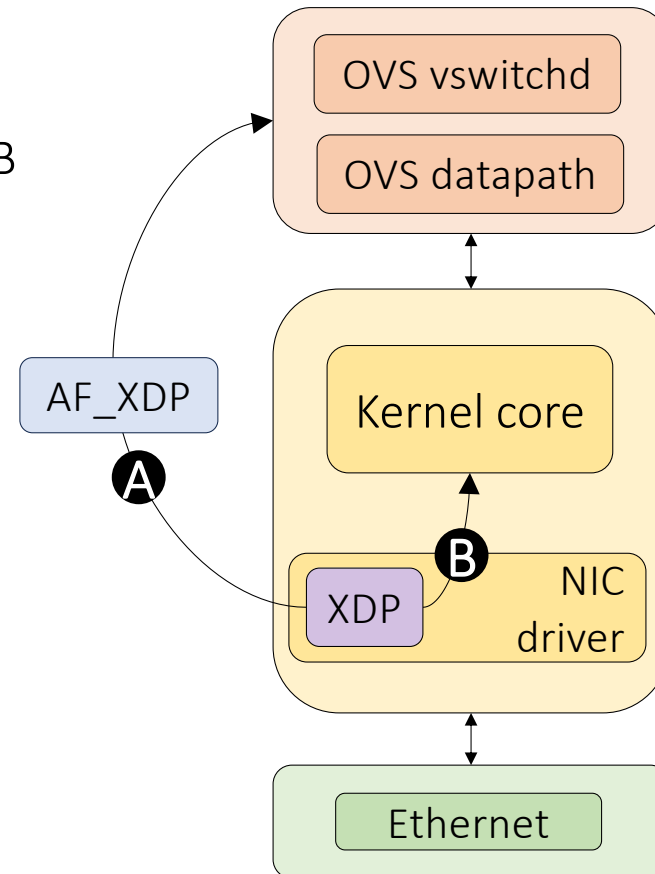
The future of OVS?

- The OVS datapath runs in user space, avoiding kernel constraints
- There are no inherent limitations on processing complexity
- Provides performance comparable to OVS-DPDK for VM networking
- Outperforms OVS-DPDK for container networking



The future of OVS?

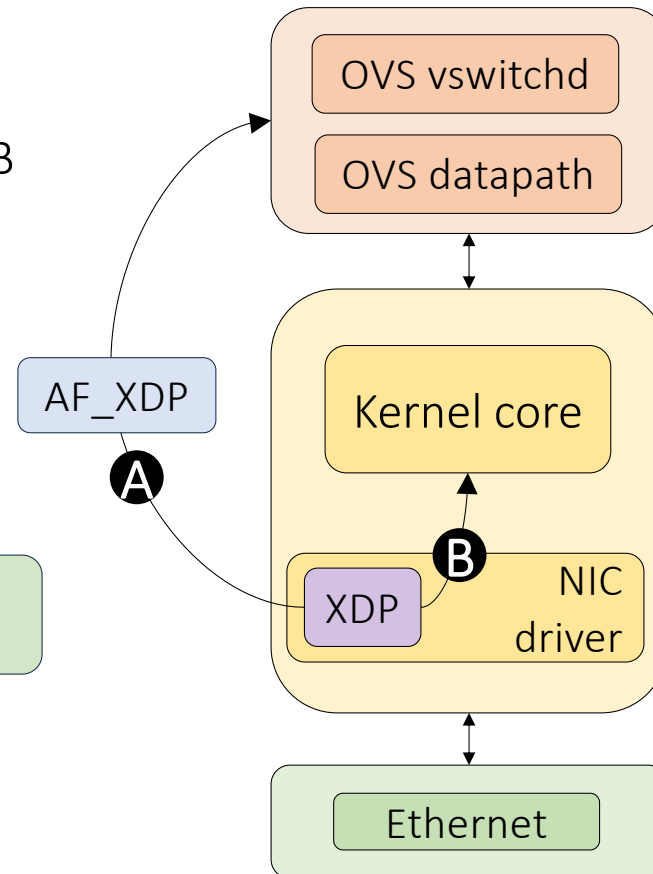
- For container networking packets can follow path B
- The XDP program passes them to the kernel core
 - Packets go through GRO, Netfilter and then bridge
 - To finally reach container's veth



The future of OVS?

- For container networking packets can follow path B
- The XDP program passes them to the kernel core
 - Packets go through GRO, Netfilter and then bridge
 - To finally reach container's veth

No need to go through the datapath in user space and pay the memory copies as in the DPDK case!

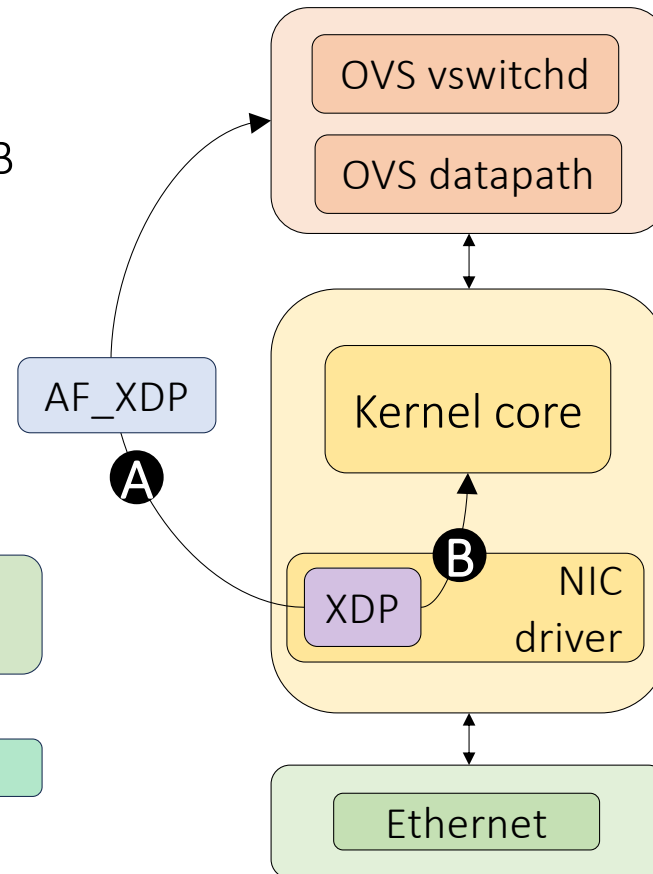


The future of OVS?

- For container networking packets can follow path B
- The XDP program passes them to the kernel core
 - Packets go through GRO, Netfilter and then bridge
 - To finally reach container's veth

No need to go through the datapath in user space and pay the memory copies as in the DPDK case!

OVS is not the only virtual switch over there...



Triton

Triton: A Flexible Hardware Offloading Architecture for Accelerating Apsara vSwitch in Alibaba Cloud

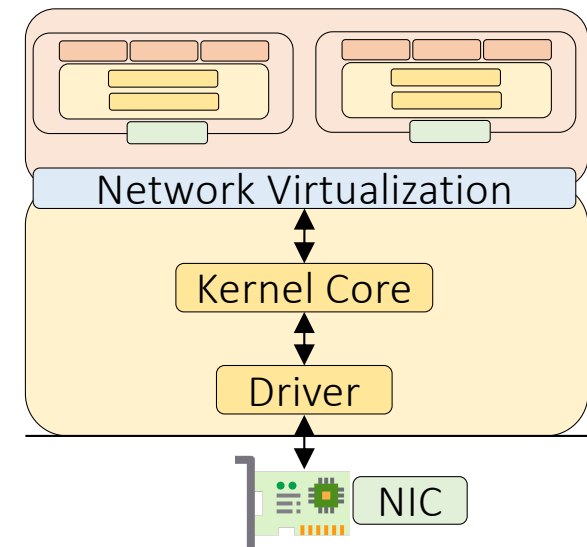
Xing Li^{†§*}, Xiaochong Jiang^{†*}, Ye Yang^{§*}, Lilong Chen[†], Yi Wang[§], Chao Wang[§], Chao Xu[§], Yilong Lv[§], Bowen Yang[§], Taotao Wu[§], Haifeng Gao[§], Zikang Chen[§], Yisong Qiao[§], Hongwei Ding[§], Yijian Dong[§], Hang Yang[§], Jianming Song[§], Jianyuan Lu[§], Pengyu Zhang[§], Chengkun Wei^{†□}, Zihui Zhang[†], Wenzhi Chen^{†□}, Qinming He[†], Shunmin Zhu^{‡§□}

[†]Zhejiang University [§]Alibaba Cloud [‡]Tsinghua University
alibaba_cloud_network@alibaba-inc.com

- A solution proposed by Alibaba and running in production in their datacenters
- A hardware acceleration for the Apsara vSwitch (AVS)
- Work presented at ACM SIGCOMM 2024 (Sidney)

Apsara vSwitch

- Apsara vSwitch (AVS) is Alibaba's proprietary virtual switch, part of the Achelous network virtualization platform
- It runs on each host to handle network connectivity for virtual machines, containers, and bare-metal instances at cloud scale
- AVS is tailored to Alibaba's internal workloads and is not open source



Evolution of AVS

- AVS 1.0
 - Pure software dataplane implemented using Linux kernel modules
 - Monolithic packet-processing pipeline
 - Strong dependence on the Linux networking stack

Evolution of AVS

- AVS 1.0
 - Pure software dataplane implemented using Linux kernel modules
 - Monolithic packet-processing pipeline
 - Strong dependence on the Linux networking stack
- AVS 2.0
 - Introduction of fast-path / slow-path separation
 - Flow-based caching to bypass control-plane processing on cache hits
 - Minimal kernel fast path, reducing reliance on complex Linux kernel networking subsystems

Evolution of AVS

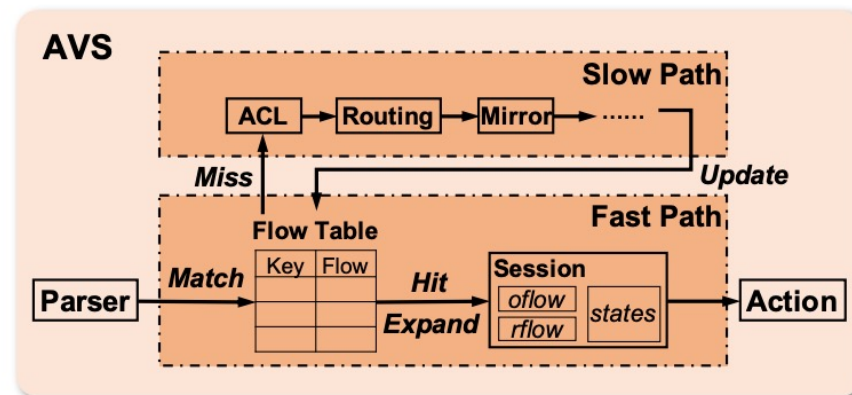
- AVS 1.0
 - Pure software dataplane implemented using Linux kernel modules
 - Monolithic packet-processing pipeline
 - Strong dependence on the Linux networking stack

Does this remind you something?

- AVS 2.0
 - Introduction of fast-path / slow-path separation
 - Flow-based caching to bypass control-plane processing on cache hits
 - Minimal kernel fast path, reducing reliance on complex Linux kernel networking subsystems

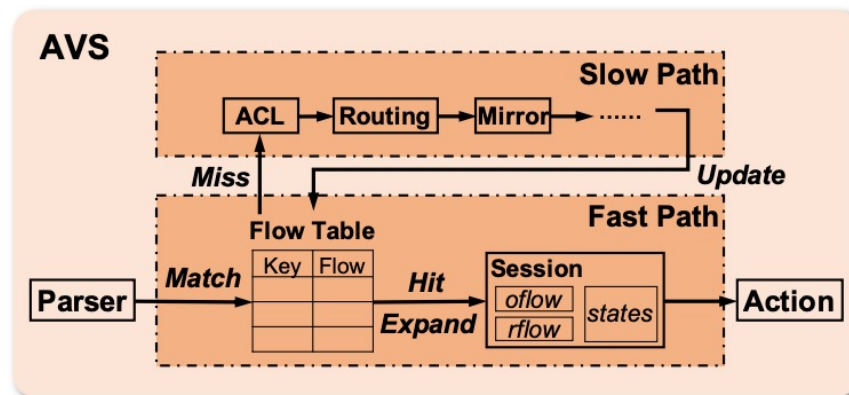
Evolution of AVS

- AVS 3.0
 - Fully user space dataplane built on DPDK
 - Kernel bypass for packet I/O



Evolution of AVS

- AVS 3.0
 - Fully user space dataplane built on DPDK
 - Kernel bypass for packet I/O

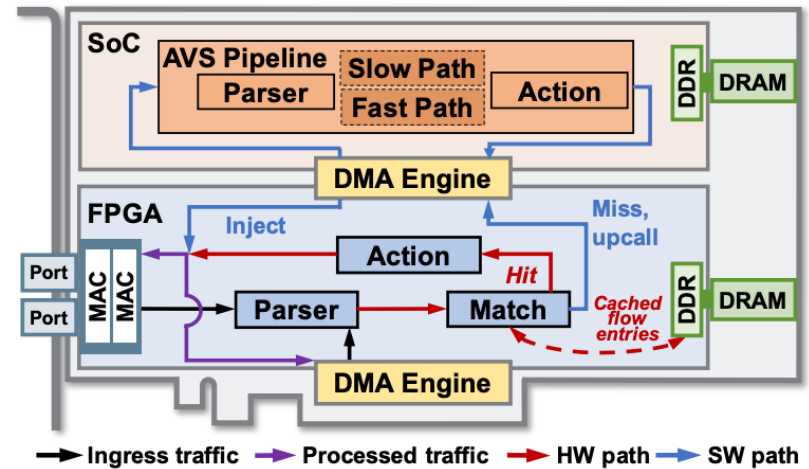


High host CPU overhead to get performance
(many CPU cores needed) ☹

AVS competes with tenant VMs for shared
resources, particularly memory bandwidth and
cache capacity, leading to contention

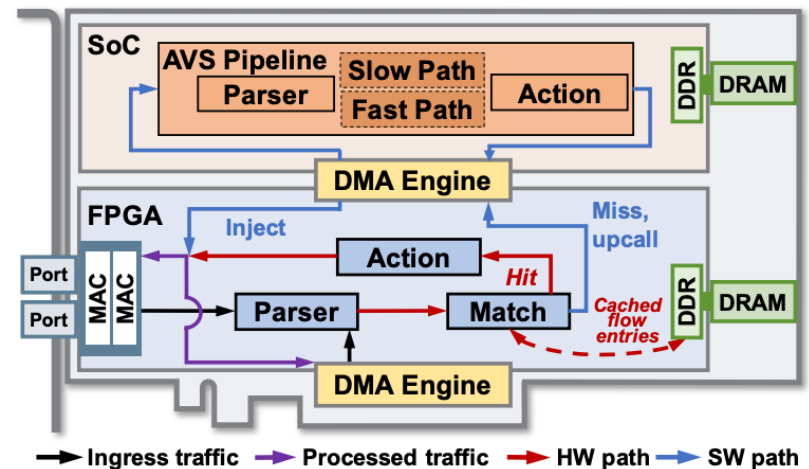
Evolution of AVS

- AVS 3.0 (Sep-Path)
 - First hardware-offloaded architecture for AVS
 - NIC SoC executes the slow path (control and exception handling)
 - NIC FPGA implements the fast path for high-throughput packet processing



Evolution of AVS

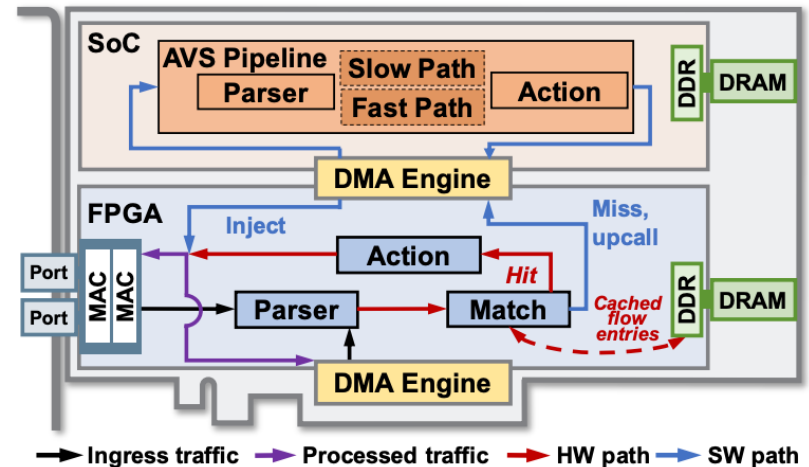
- AVS 3.0 (Sep-Path)
 - First hardware-offloaded architecture for AVS
 - NIC SoC executes the slow path (control and exception handling)
 - NIC FPGA implements the fast path for high-throughput packet processing



Similar architecture of OVS: packets that do not match any entry in the fast path are sent to the NIC SoC

Evolution of AVS

- AVS 3.0 (Sep-Path)
 - First hardware-offloaded architecture for AVS
 - NIC SoC executes the slow path (control and exception handling)
 - NIC FPGA implements the fast path for high-throughput packet processing



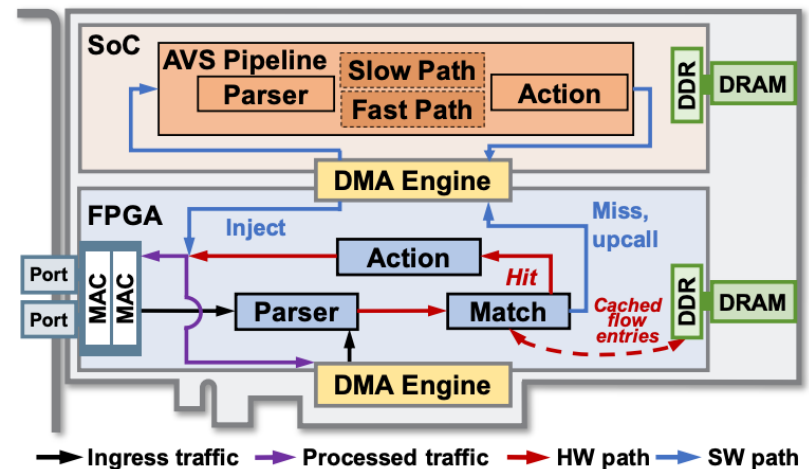
Similar architecture of OVS: packets that do not match any entry in the fast path are sent to the NIC SoC

After several years of deployment, this architecture brought few issues...

Deployment issues with Sep-Path

Issue 1: unpredictable performance

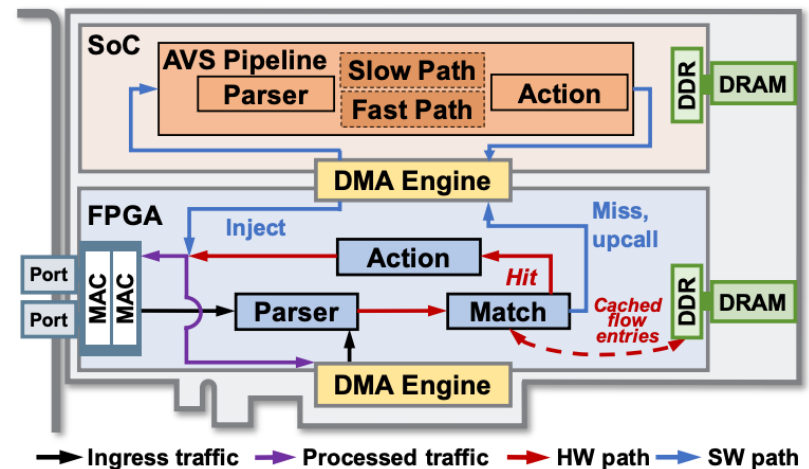
- Hardware data path has finite state capacity
 - Excess traffic falls back to software forwarding, increasing CPU contention and degrading tenant performance
- Offload benefits are highly skewed
 - Most tenants generate short-lived flows that remain on the software path



Deployment issues with Sep-Path

Issue 2: decreased iteration velocity

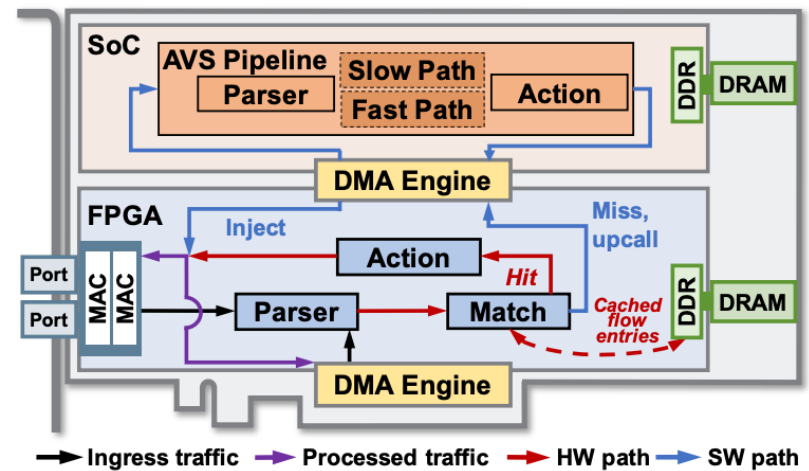
- Cloud platforms require continuous feature integration and bug fixes
- This architecture forces parallel changes in both software and hardware paths, increasing engineering effort
- Release frequency dropped from 4/5 releases/year to 2/3



Deployment issues with Sep-Path

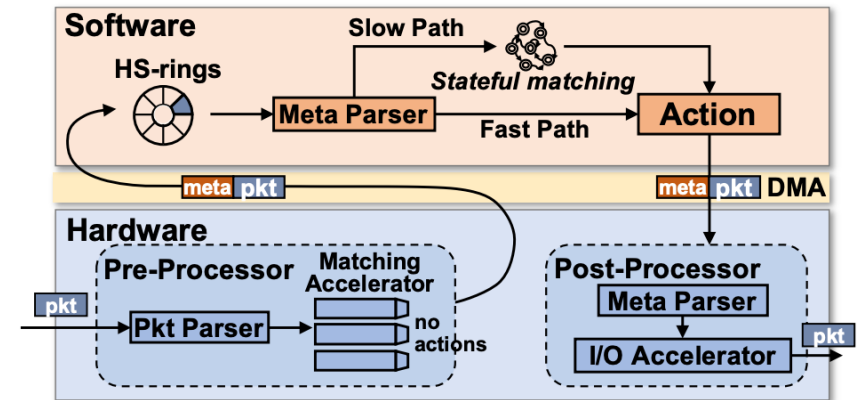
Issue 3: high maintenance costs

- Debugging complexity significantly increases
- Cross-path interaction is a major source of problems
 - 25% of issues caused by hardware
 - 40% caused by software-hardware interaction
 - 35% caused by AVS



Triton

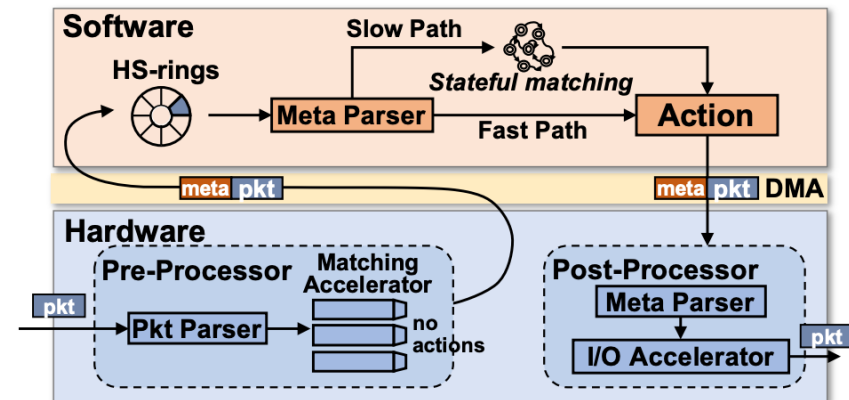
- Unified serial data path: All packets follow the same pipeline: no split hardware/software paths
- Three-stage processing model: HW Pre-Processor → Software match-action → HW Post-Processor
- Function-driven partitioning: Hardware executes fixed, high-cost primitives; software handles flexible, evolving logic



Triton: advantages

1: Predictable performance

- Single forwarding path eliminated traffic skew and cross-path imbalance



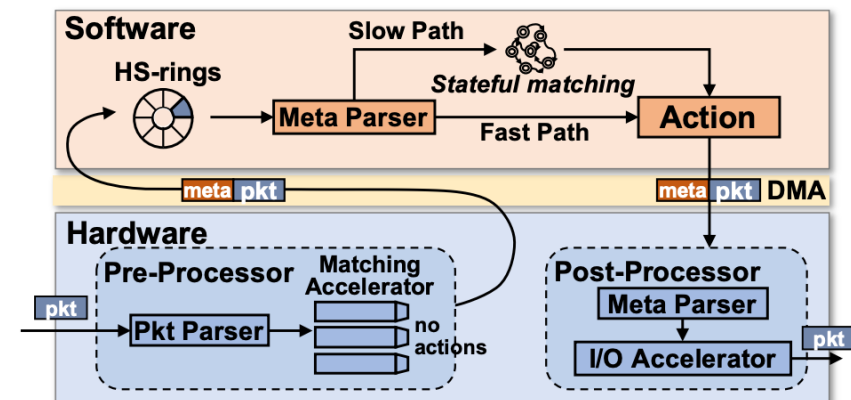
Triton: advantages

1: Predictable performance

- Single forwarding path eliminated traffic skew and cross-path imbalance

2: Improved development agility

- No parallel hw/sw feature implementation; reduces engineering overhead and speeds up iteration



Triton: advantages

1: Predictable performance

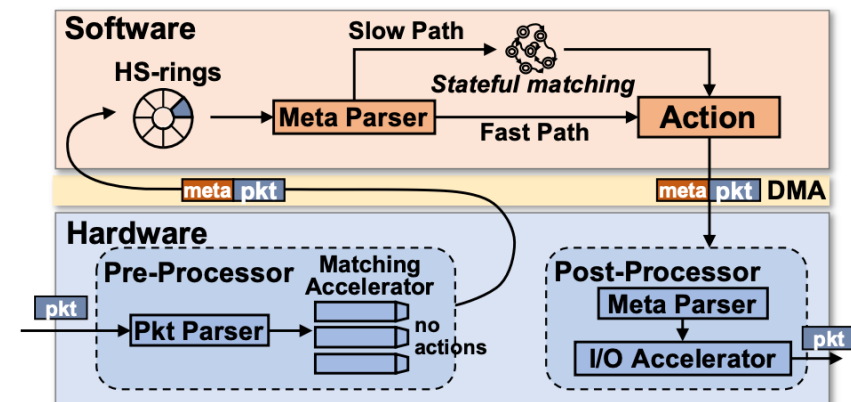
- Single forwarding path eliminated traffic skew and cross-path imbalance

2: Improved development agility

- No parallel hw/sw feature implementation; reduces engineering overhead and speeds up iteration

3: Simplified operations and debugging

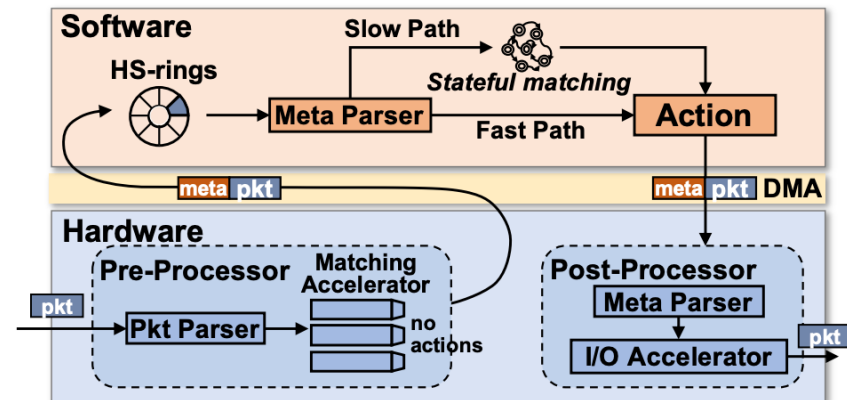
- Removes hw/sw state divergence and complex synchronization issues



Triton: advantages

4: Better workload control

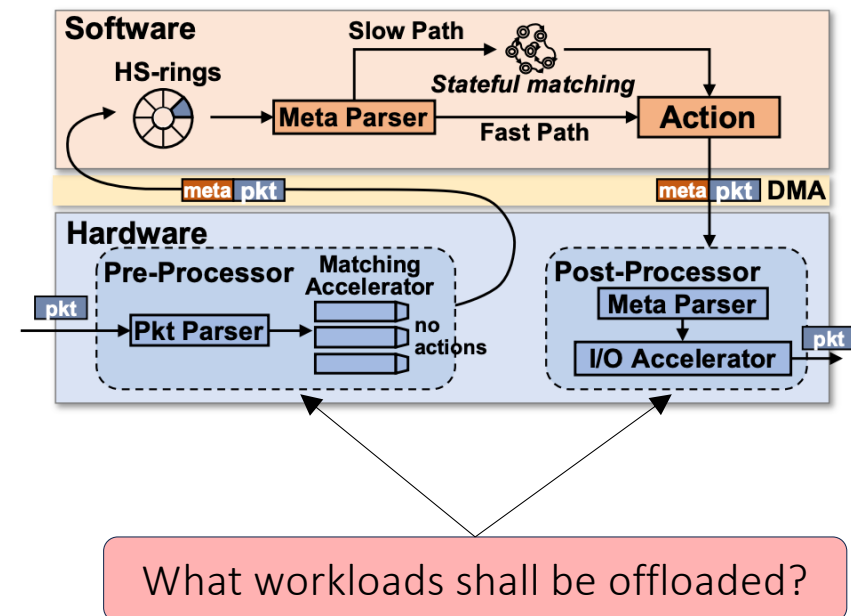
- Stage-level partitioning enables precise performance modelling and balanced hw/sw utilization



Triton: advantages

4: Better workload control

- Stage-level partitioning enables precise performance modelling and balanced hw/sw utilization



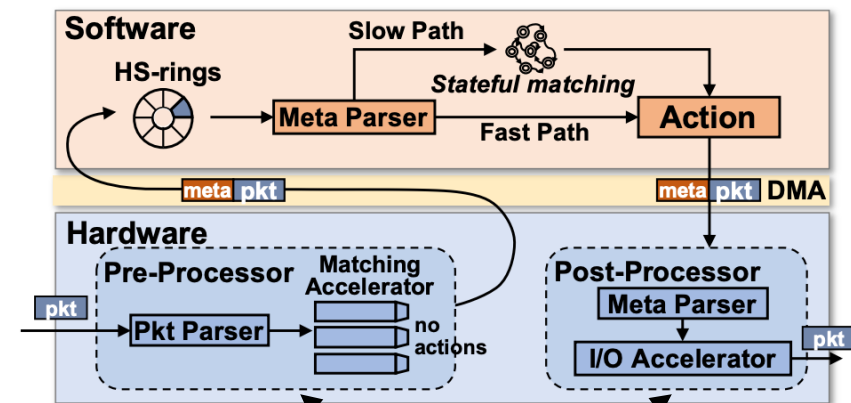
Triton: advantages

4: Better workload control

- Stage-level partitioning enables precise performance modelling and balanced hw/sw utilization

Stage	Cost (CPU usage)	Workload Distribution
Parsing	27.36%	Hardware
Matching	11.2%	Software & Hardware assisted
Action	24.32%	Software & Hardware assisted
Driver	29.85%	Software & Hardware assisted
Statistics	7.17%	Software

Principle: putting the flexible workloads in software and keep fixed but time-consuming tasks in hardware



What workloads shall be offloaded?

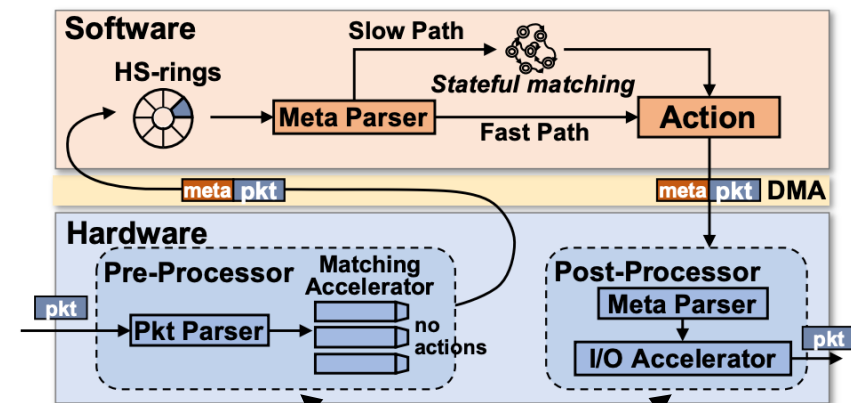
Triton: advantages

4: Better workload control

- Stage-level partitioning enables precise performance modelling and balanced hw/sw utilization

Stage	Cost (CPU usage)	Workload Distribution
Parsing	27.36%	Hardware
Matching	11.2%	Software & Hardware assisted
Action	24.32%	Software & Hardware assisted
Driver	29.85%	Software & Hardware assisted
Statistics	7.17%	Software

Principle: putting the flexible workloads in software and keep fixed but time-consuming tasks in hardware



What workloads shall be offloaded?

...but software now become the new bottleneck... ☹

Triton: optimizations

1: Jumbo frames support

- This increases the effective payload per packet, reducing performance pressure on forwarding components

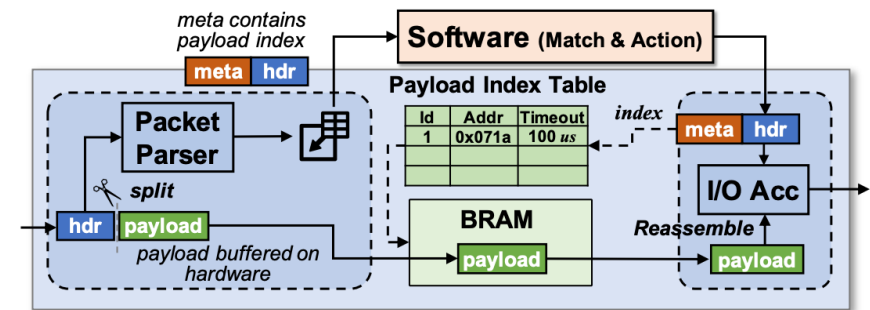
Triton: optimizations

1: Jumbo frames support

- This increases the effective payload per packet, reducing performance pressure on forwarding components

2: Header-payload slicing

- This is to mitigate PCIe bandwidth usage caused by needless data movement between software and hardware



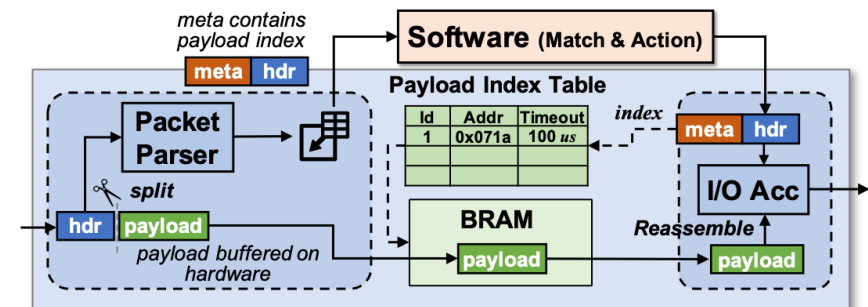
Triton: optimizations

1: Jumbo frames support

- This increases the effective payload per packet, reducing performance pressure on forwarding components

2: Header-payload slicing

- This is to mitigate PCIe bandwidth usage caused by needless data movement between software and hardware



This reduces PCIe bandwidth consumption during DMA operations: when forwarding a jumbo frame, it can save approximately 97% of the PCIe bandwidth

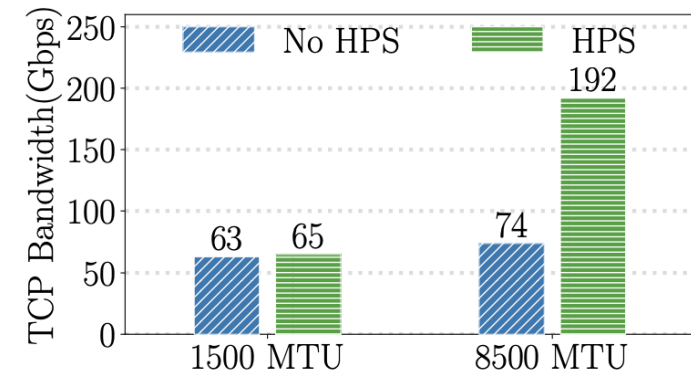
Triton: optimizations

1: Jumbo frames support

- This increases the effective payload per packet, reducing performance pressure on forwarding components

2: Header-payload slicing

- This is to mitigate PCIe bandwidth usage caused by needless data movement between software and hardware



Using jumbo frames together with HPS reduces unnecessary data movement by more than 80%, resulting in up to a 3× increase in bandwidth.

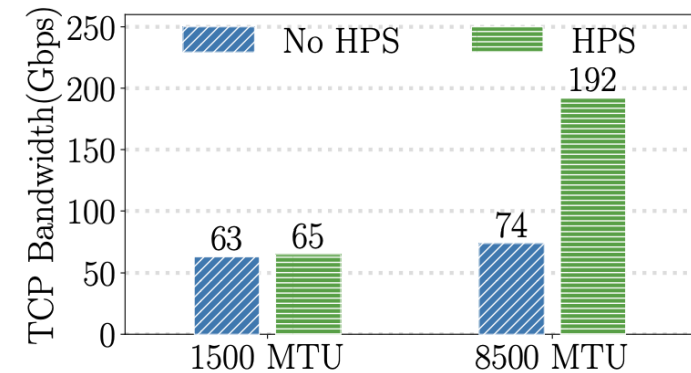
Triton: optimizations

1: Jumbo frames support

- This increases the effective payload per packet, reducing performance pressure on forwarding components

2: Header-payload slicing

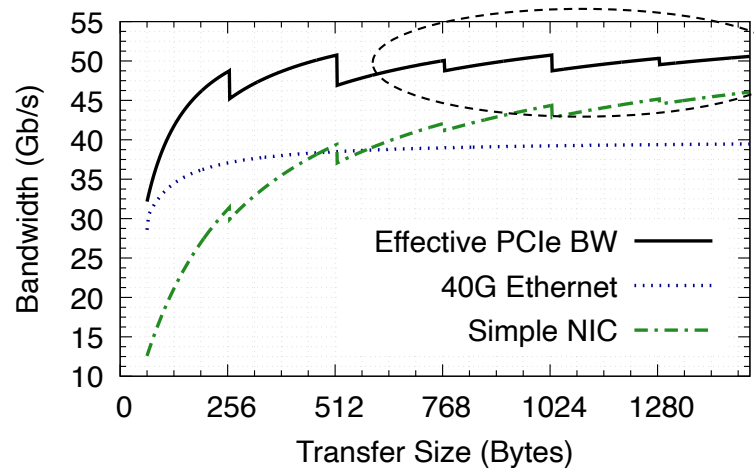
- This is to mitigate PCIe bandwidth usage caused by needless data movement between software and hardware



Using jumbo frames together with HPS reduces unnecessary data movement by more than 80%, resulting in up to a 3× increase in bandwidth.

This is all great, but PCIe imposes higher overheads if you transfer small amount of data each time...

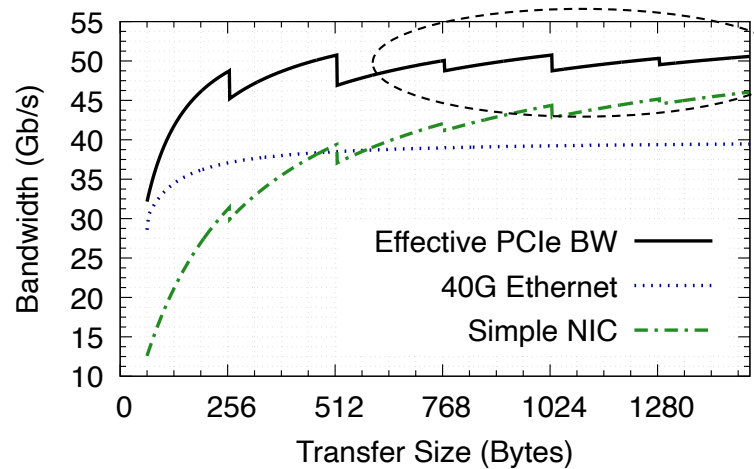
Triton: optimizations



This why we said in a past lecture that TSO/UFO/LRO are useful 😊

This is all great, but PCIe imposes higher overheads if you transfer small amount of data each time...

Triton: optimizations



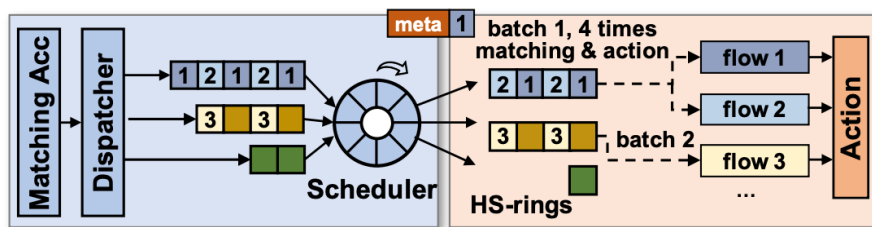
This why we said in a past lecture that TSO/UFO/LRO are useful 😊

Ideas? 😊

This is all great, but PCIe imposes higher overheads if you transfer small amount of data each time...

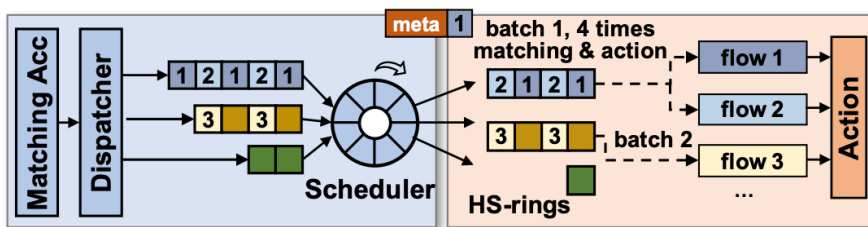
Triton: optimizations

You can do batch processing!



Triton: optimizations

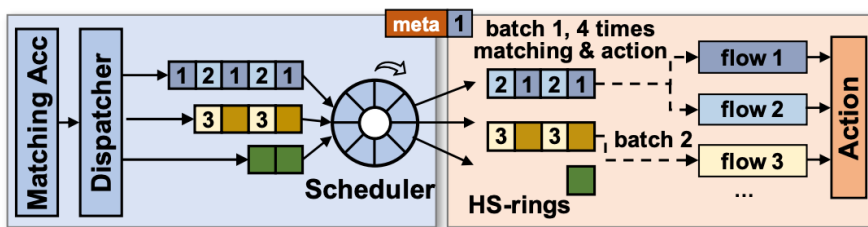
You can do batch processing!



When processing a batch of packet headers, the CPU executes the match-action logic sequentially for each one. This offers limited performance benefits, as repeated table lookups can still lead to instruction and data cache misses

Triton: optimizations

You can do batch processing!

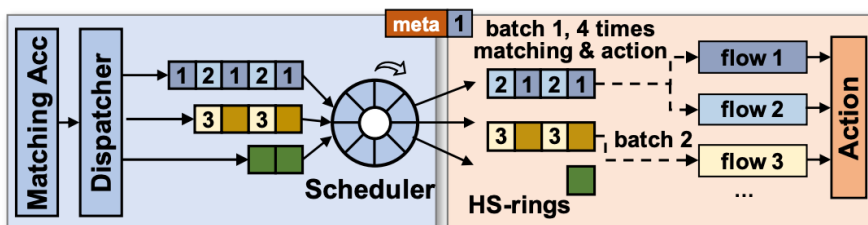


When processing a batch of packet headers, the CPU executes the match-action logic sequentially for each one. This offers limited performance benefits, as repeated table lookups can still lead to instruction and data cache misses

Ideas?

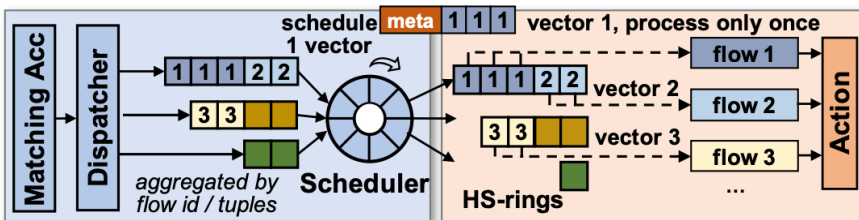
Triton: optimizations

You can do batch processing!



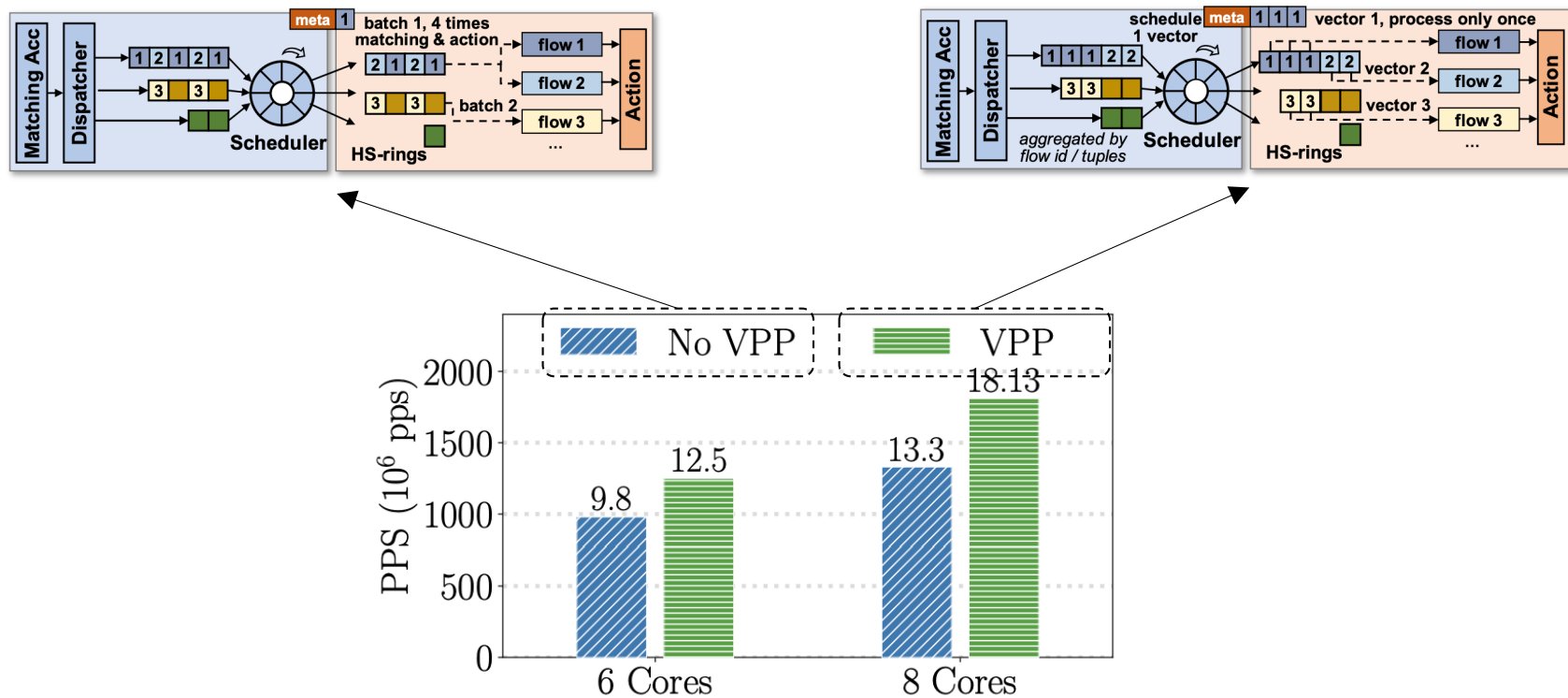
When processing a batch of packet headers, the CPU executes the match-action logic sequentially for each one. This offers limited performance benefits, as repeated table lookups can still lead to instruction and data cache misses

Ideas?

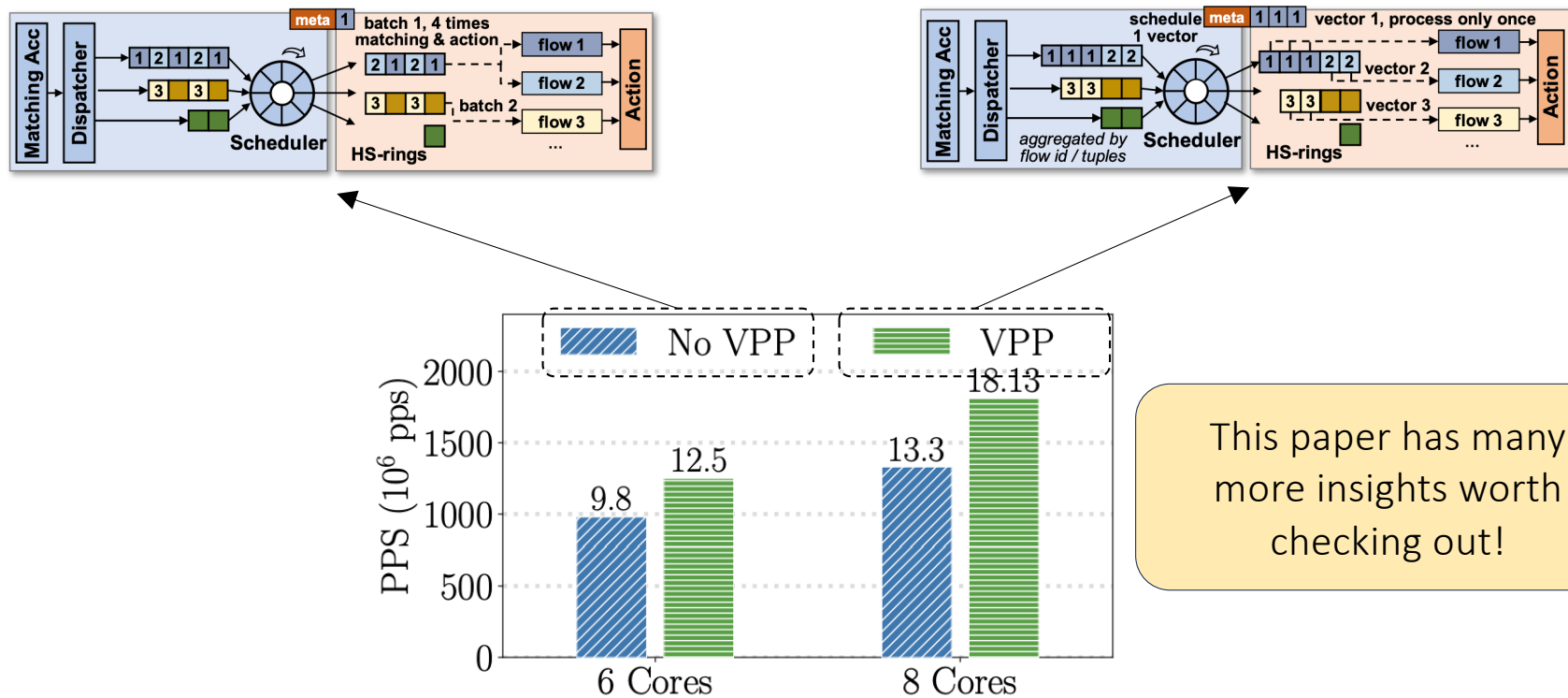


Flow-based aggregation in hardware so that cache-hit rate in software is greatly improved!

Triton: optimizations



Triton: optimizations



This paper has many more insights worth checking out!

Takeaways

- Virtual switches are a key component in today's cloud stacks

Takeaways

- Virtual switches are a key component in today's cloud stacks
- They can implement packet-processing operations just in software (i.e., OVS)
 - They can use an in-kernel component (i.e., kernel module or eBPF)
 - They can leverage kernel-bypass mechanisms (i.e., AF_XDP)

Takeaways

- Virtual switches are a key component in today's cloud stacks
- They can implement packet-processing operations just in software (i.e., OVS)
 - They can use an in-kernel component (i.e., kernel module or eBPF)
 - They can leverage kernel-bypass mechanisms (i.e., AF_XDP)
- ...or use a combination of ad-hoc hardware and software (i.e., Triton)

Takeaways

- Virtual switches are a key component in today's cloud stacks
- They can implement packet-processing operations just in software (i.e., OVS)
 - They can use an in-kernel component (i.e., kernel module or eBPF)
 - They can leverage kernel-bypass mechanisms (i.e., AF_XDP)
- ...or use a combination of ad-hoc hardware and software (i.e., Triton)
- ...or implement packet-processing fully in hardware, inside the NIC (i.e., AccelNet)

Azure Accelerated Networking: SmartNICs in the Public Cloud

Daniel Firestone Andrew Putnam Sambhrama Mundkur Derek Chiou Alireza Dabagh
Mike Andrewartha Hari Angepat Vivek Bhanu Adrian Caulfield Eric Chung
Harish Kumar Chandrappa Somesh Chaturmohta Matt Humphrey Jack Lavier Norman Lam
Fengfen Liu Kalin Ovtcharov Jitu Padhye Gautham Popuri Shachar Raindel Tejas Sapre
Mark Shaw Gabriel Silva Madhan Sivakumar Nisheet Srivastava Anshuman Verma Qasim Zuhair
Deepak Bansal Doug Burger Kushagra Vaid David A. Maltz Albert Greenberg
Microsoft

For the more curious ones..

- OVS is not the only open-source virtual switch around...

For the more curious ones..

- OVS is not the only open-source virtual switch around...
- BESS: Berkeley Extensible Software Switch

BESS

Berkeley Extensible Software Switch

Modern NICs implement various features in hardware, such as protocol offloading, multicore supports, traffic control, and self virtualization. This approach exposes several issues: protocol dependence, limited hardware resources, and incomplete/buggy/non-compliant implementation. Even worse, the slow evolution of hardware NICs due to increasingly overwhelming design complexity cannot keep up in time with the new protocols and rapidly changing network architectures. We introduce the SoftNIC architecture to fill the gap between hardware capabilities and user demands. Our current SoftNIC prototype implements sophisticated NIC features on a few dedicated processor cores, while assuming only streamlined functionalities in hardware. The preliminary evaluation results show that most NIC features can be implemented in software with minimum performance cost, while the flexibility of software provides further potential benefits.

[Read the Paper](#)[See the Code](#)

For the more curious ones..

- OVS is not the only open-source virtual switch around...
- BESS: Berkeley Extensible Software Switch
- Snabb Switch

BESS

Berkeley Extensible Software Switch

Modern NICs implement various features in hardware, such as protocol offloading, multicore supports, traffic control, and self virtualization. This approach exposes several issues: protocol dependence, limited hardware resources, and incomplete/buggy/non-compliant implementation. Even worse, the slow evolution of hardware NICs due to increasingly overwhelming design complexity cannot keep up in time with the new protocols and rapidly changing network architectures. We introduce the SoftNIC architecture to fill the gap between hardware capabilities and user demands. Our current SoftNIC prototype implements sophisticated NIC features on a few dedicated processor cores, while assuming only streamlined functionalities in hardware. The preliminary evaluation results show that most NIC features can be implemented in software with minimum performance cost, while the flexibility of software provides further potential benefits.

[Read the Paper](#)[See the Code](#)

Snabb Switch

Snabb Switch is a simple and fast packet networking toolkit.

We are also a grassroots community of programmers and network engineers who help each other to build and deploy new network elements. We care about practical applications and finding simpler ways to do things.

The Snabb Switch community are active in [applying modern programming techniques](#), [do-it-yourself operator networking](#), [high-level device drivers](#), [fast userspace virtio networking](#), [universal SIMD protocol offloads](#), and [applying compiler technology to networking](#).

You are welcome to join our community. If you have an application that you want to build, or you want to use one that we are already developing, or you want to contribute in some other way, then please join the [snabb-devel mailing list](#) and read on.

For the more curious ones..

- OVS is not the only open-source virtual switch around...
- BESS: Berkeley Extensible Software Switch
- Snabb Switch

Snabb Switch

Snabb Switch is a simple and fast packet networking toolkit.

We are also a grassroots community of programmers and network engineers who help each other to build and deploy new network elements. We care about practical applications and finding simpler ways to do things.

The Snabb Switch community are active in [applying modern programming techniques](#), [do-it-yourself operator networking](#), [high-level device drivers](#), [fast userspace virtio networking](#), [universal SIMD protocol offloads](#), and [applying compiler technology to networking](#).

You are welcome to join our community. If you have an application that you want to build, or you want to use one that we are already developing, or you want to contribute in some other way, then please join the [snabb-devel mailing list](#) and read on.

BESS

Berkeley Extensible Software Switch

Modern NICs implement various features in hardware, such as protocol offloading, multicore supports, traffic control, and self virtualization. This approach exposes several issues: protocol dependence, limited hardware resources, and incomplete/buggy/non-compliant implementation. Even worse, the slow evolution of hardware NICs due to increasingly overwhelming design complexity cannot keep up in time with the new protocols and rapidly changing network architectures. We introduce the SoftNIC architecture to fill the gap between hardware capabilities and user demands. Our current SoftNIC prototype implements sophisticated NIC features on a few dedicated processor cores, while assuming only streamlined functionalities in hardware. The preliminary evaluation results show that most NIC features can be implemented in software with minimum performance cost, while the flexibility of software provides further potential benefits.

[Read the Paper](#)[See the Code](#)

There are more around!

For the more curious ones..

- OVS is not the only open-source virtual switch around...
- BESS: Berkeley Extensible Software Switch
- Snabb Switch

Snabb Switch

Snabb Switch is a simple and fast packet networking toolkit.

We are also a grassroots community of programmers and network engineers who help each other to build and deploy new network elements. We care about practical applications and finding simpler ways to do things.

The Snabb Switch community are active in [applying modern programming techniques](#), [do-it-yourself operator networking](#), [high-level device drivers](#), [fast userspace virtio networking](#), [universal SIMD protocol offloads](#), and [applying compiler technology to networking](#).

You are welcome to join our community. If you have an application that you want to build, or you want to use one that we are already developing, or you want to contribute in some other way, then please join the [snabb-devel mailing list](#) and read on.

BESS

Berkeley Extensible Software Switch

Modern NICs implement various features in hardware, such as protocol offloading, multicore supports, traffic control, and self virtualization. This approach exposes several issues: protocol dependence, limited hardware resources, and incomplete/buggy/non-compliant implementation. Even worse, the slow evolution of hardware NICs due to increasingly overwhelming design complexity cannot keep up in time with the new protocols and rapidly changing network architectures. We introduce the SoftNIC architecture to fill the gap between hardware capabilities and user demands. Our current SoftNIC prototype implements sophisticated NIC features on a few dedicated processor cores, while assuming only streamlined functionalities in hardware. The preliminary evaluation results show that most NIC features can be implemented in software with minimum performance cost, while the flexibility of software provides further potential benefits.

[Read the Paper](#)[See the Code](#)

There are more around!

Have fun with them 😊